

Bridge

ARDUINO

UNO

Q

begin()



call()



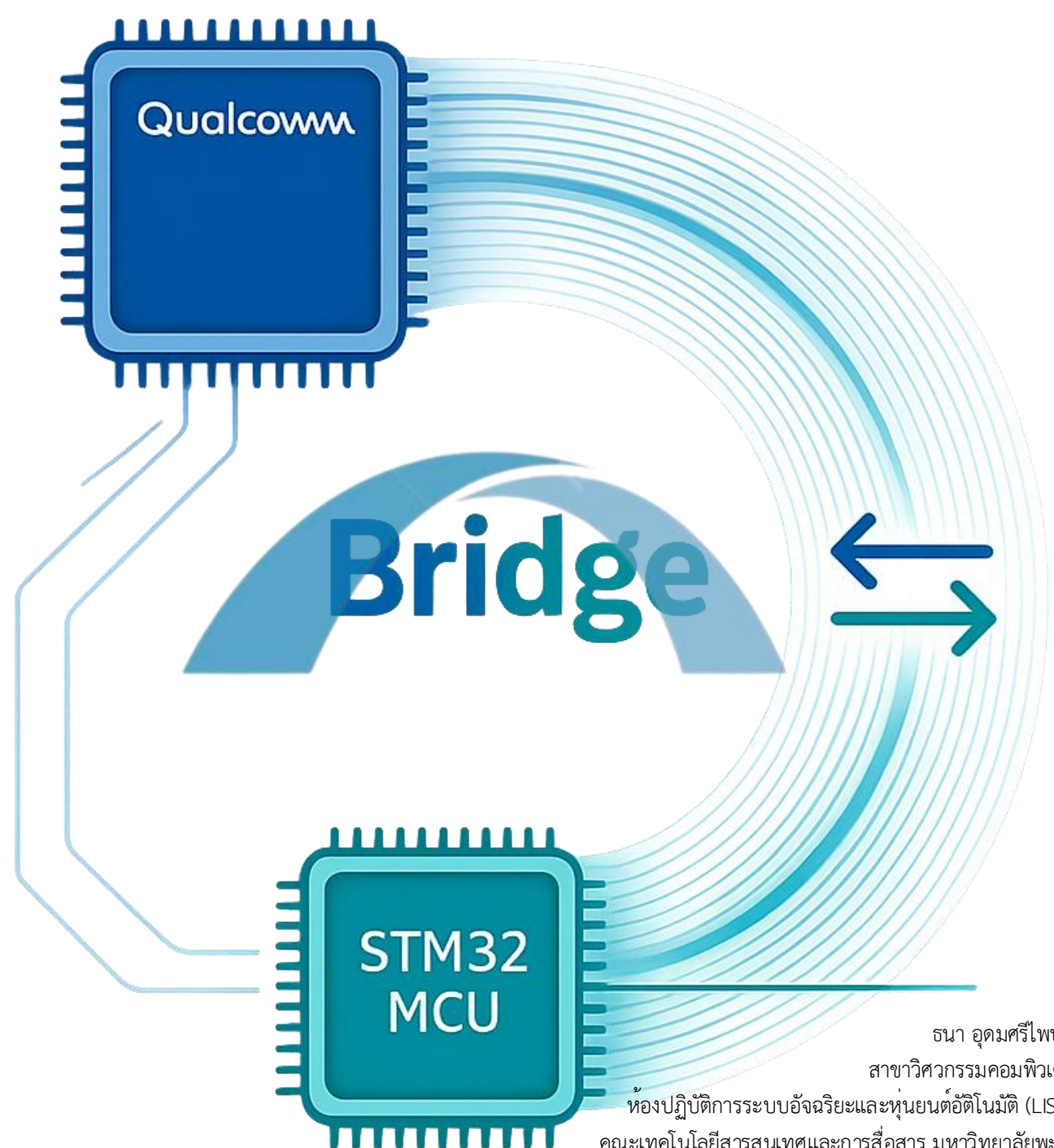
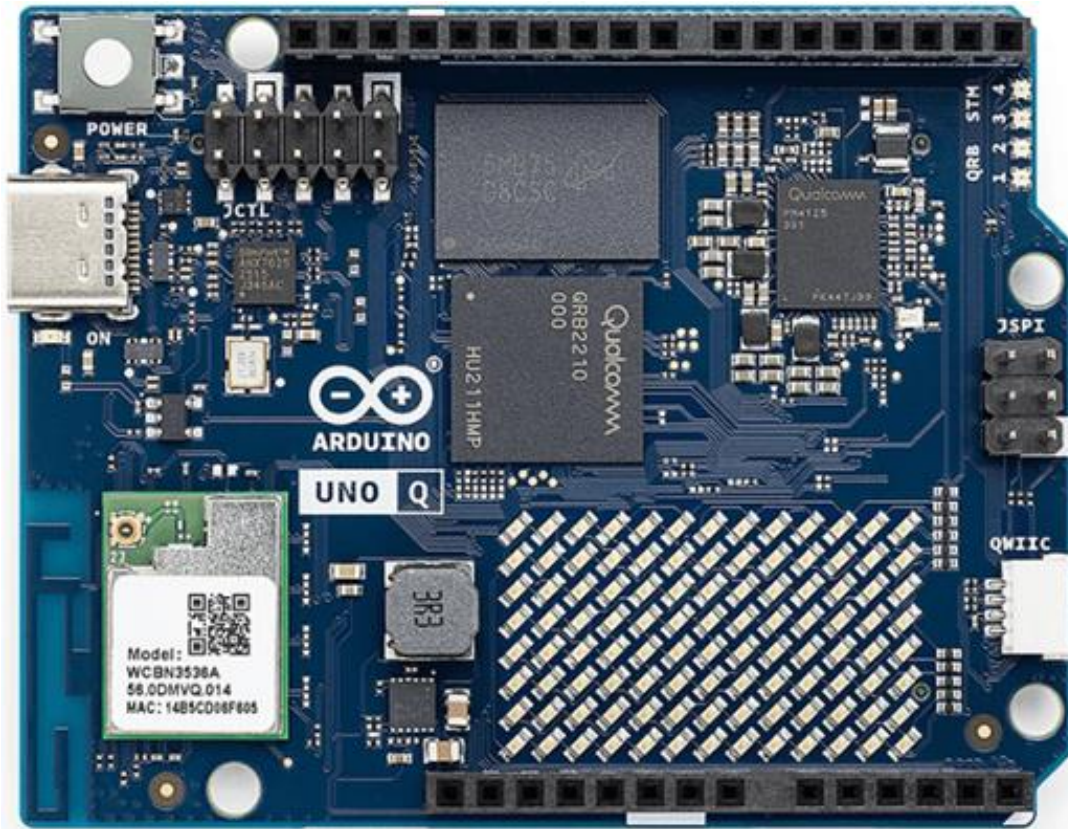
notify()



provide()



provide_safe()



ธนา อุดมศรีไพบุลย์

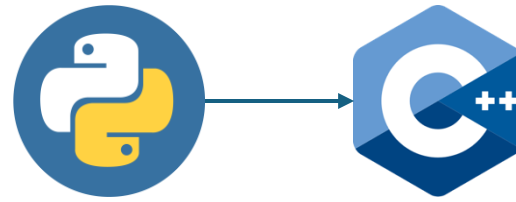
สาขาวิศวกรรมคอมพิวเตอร์

ห้องปฏิบัติการระบบอัจฉริยะและหุ่นยนต์อัตโนมัติ (LISAR)

คณะเทคโนโลยีสารสนเทศและการสื่อสาร มหาวิทยาลัยพะเยา

Bridge Library ซึ่งเป็นกลไกการสื่อสารแบบ **Remote Procedure Call (RPC)** ที่ใช้เชื่อมต่อระหว่างหน่วยประมวลผล **Qualcomm** และไมโครคอนโทรลเลอร์ **STM32** บนบอร์ด **Arduino UNO Q** ระบบนี้ช่วยให้โปรเซสเซอร์ทั้งสองฝั่งสามารถเรียกใช้งานฟังก์ชันของกันและกันได้เสมือนทำงานอยู่บนเครื่องเดียวกัน โดยมี **Arduino Router** ทำหน้าที่เป็นตัวกลางควบคุมการรับส่งข้อมูลผ่านโครงสร้างเครือข่ายแบบ **Star Topology** และโปรโตคอล **MessagePack RPC** คุณสมบัติเด่นของระบบคือรองรับการสื่อสารแบบหลายจุด ซึ่งช่วยให้แอปพลิเคชันบน **Linux** และ **MCU** ทำงานร่วมกันได้อย่างซับซ้อนและมีประสิทธิภาพ ทั้งนี้ผู้ใช้งานจำเป็นต้องหลีกเลี่ยงการเข้าถึงทรัพยากรฮาร์ดแวร์บางส่วนโดยตรงเพื่อป้องกันความล้มเหลวของระบบสื่อสาร โดยรวมแล้วเทคโนโลยีนี้ช่วยลดความยุ่งยากในการพัฒนาโปรเจกต์ที่ต้องผสมผสานระหว่างการประมวลผลระดับสูงและการควบคุมอุปกรณ์ฮาร์ดแวร์แบบเรียลไทม์

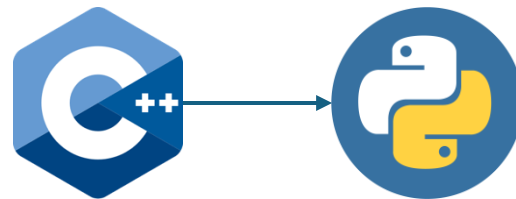
Bridge



Call (เจ้าหน้าที่)

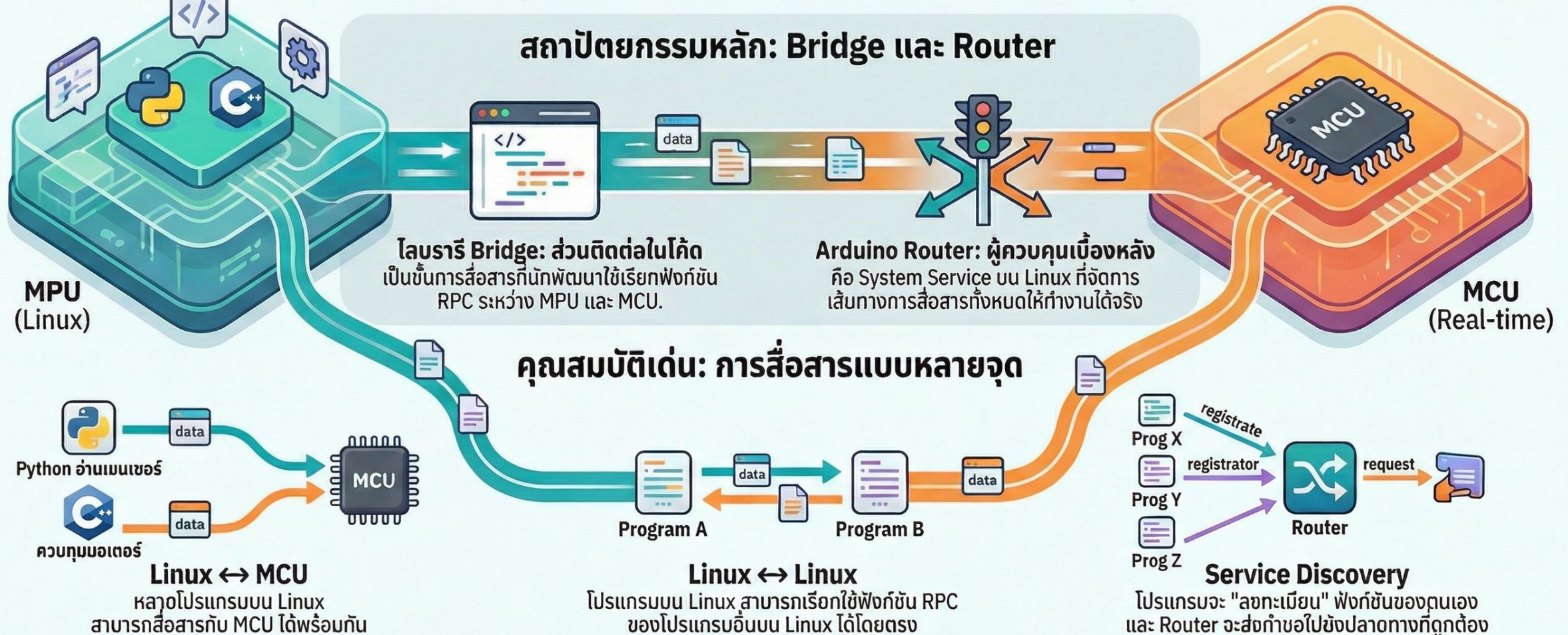


Provide (ลูกค้า)



Arduino UNO Q: เชื่อมต่อสองโปรเซสเซอร์ด้วย Bridge Library

บอร์ด Arduino UNO Q ใช้ไลบรารี Bridge ซึ่งเป็นกลไก Remote Procedure Call (RPC) เพื่อให้โปรเซสเซอร์ฝั่ง Linux (MPU) และไมโครคอนโทรลเลอร์ (MCU) สามารถแลกเปลี่ยนข้อมูลและเรียกใช้ฟังก์ชันของกันและกันได้อย่างราบรื่น เหมือนทำงานอยู่บนเครื่องเดียวกัน



ข้อควรระวังสำหรับนักพัฒนา



ห้ามใช้พอร์ตอนุกรมที่ถูกจองไว้

ห้ามเข้าถึง `/dev/ttyHS1` (ฝั่ง Linux) และ `Serial1` (ฝั่ง MCU) โดยตรง



ระวัง Deadlock

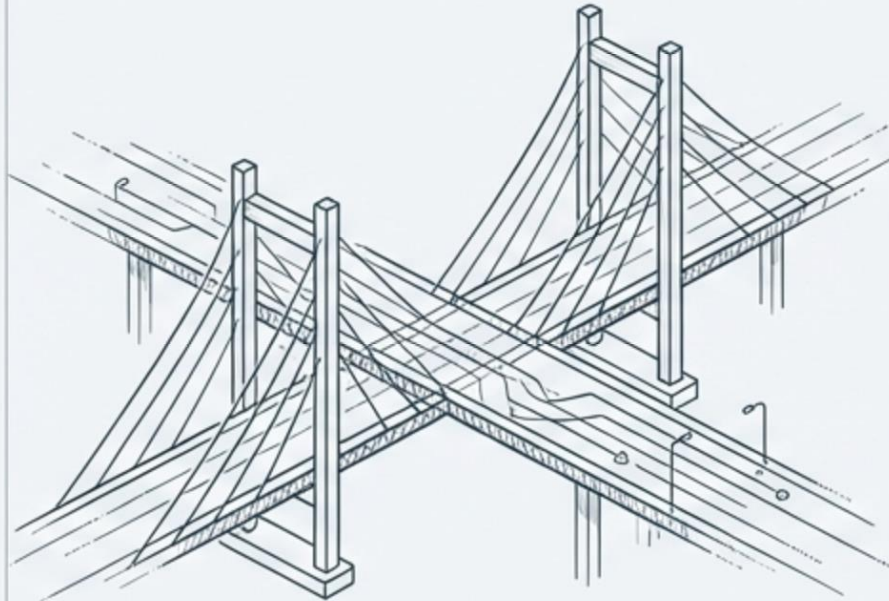
ห้ามใช้คำสั่ง `Bridge.call()` ภายในฟังก์ชันที่ถูกประกาศด้วย `provide()`

โจทย์ท้าทาย: สองโลกที่แตกต่างกันในหนึ่งเดียว

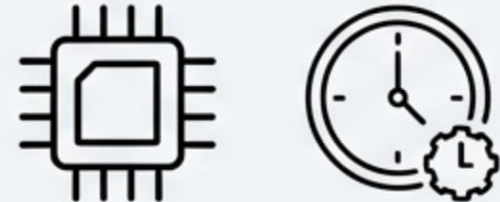
MPU (Linux)



ระบบปฏิบัติการเต็มรูปแบบ,
พลังประมวลผลสูง,
เหมาะสำหรับงานซับซ้อน
(High-level applications,
complex logic).



MCU (Zephyr RTOS)



การทำงานแบบเรียลไทม์,
ความแม่นยำด้านเวลา,
ควบคุมฮาร์ดแวร์โดยตรง
(Time-critical tasks, direct
hardware control).

เราจะทำให้โปรเซสเซอร์ทั้งสองฝั่งแลกเปลี่ยนข้อมูลกันได้อย่างไรรอยต่อ
เสมือนเป็นหนึ่งเดียวกัน?

คำตอบคือสถาปัตยกรรมใหม่: Router และ Bridge

Arduino UNO Q ใช้กลไก Remote Procedure Call (RPC) ที่ช่วยให้โปรเซสเซอร์ตัวหนึ่งสามารถเรียกใช้ฟังก์ชันของอีกตัวหนึ่งได้อย่างโปร่งใส เหมือนเป็นฟังก์ชันในเครื่องเดียวกัน



- **Router:** คือ **สมอง** ที่ควบคุมทิศทางและตรรกะการสื่อสารทั้งหมด เป็นผู้จัดการระบบ
- **Bridge:** คือ **สะพาน** หรือ API ที่เชื่อมต่อโค้ดของคุณในโลกที่ต่างกัน (MCU vs App) ให้คุยกับ Router ได้



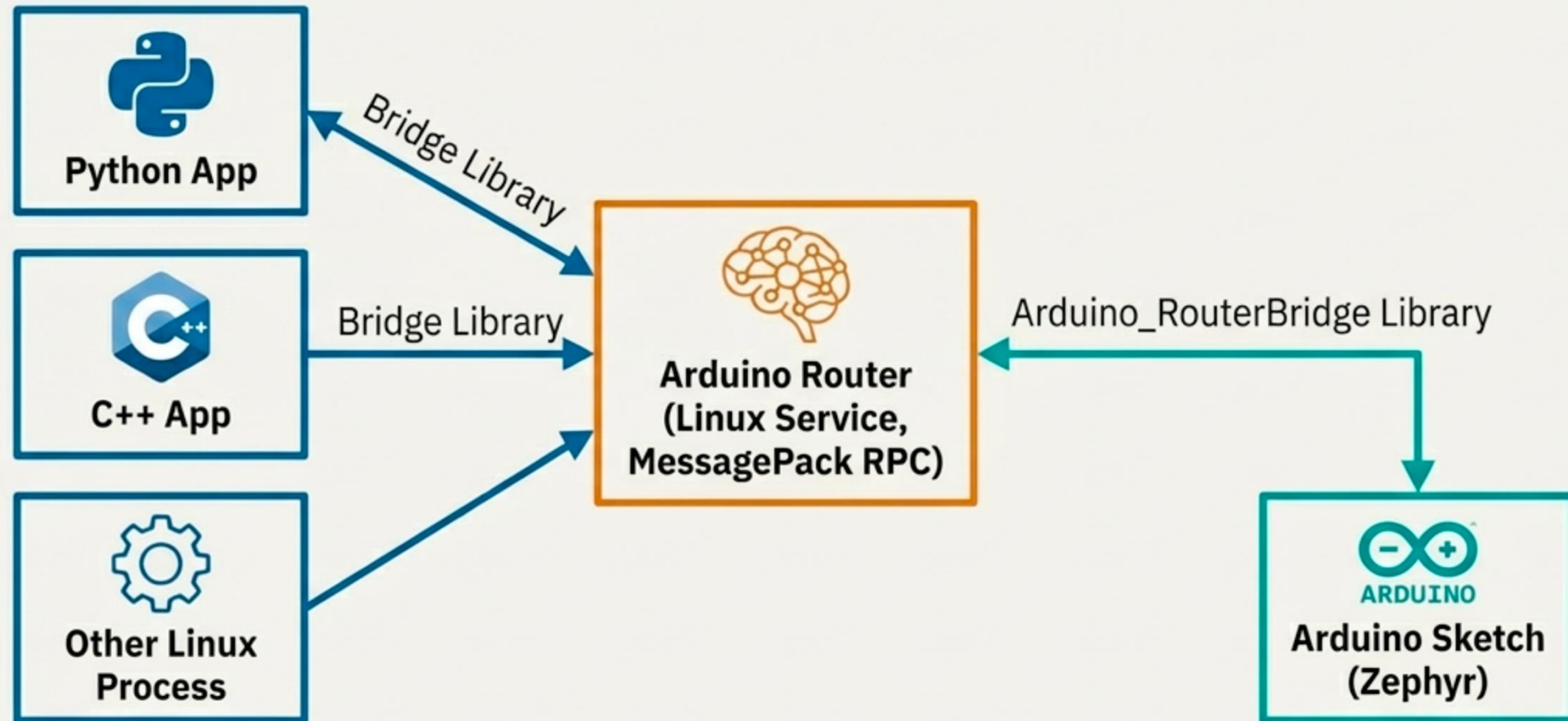
Arduino Router (ศูนย์ควบคุมการจราจร - The Traffic Control Center)

บริการเบื้องหลังบน Linux (`arduino-router`) ที่ทำหน้าที่เป็น 'สมอง' จัดการเส้นทางข้อมูลทั้งหมด

Bridge Library (สะพานเชื่อม - The Connecting Bridge)

ไลบรารีที่คุณใช้งานในโค้ด (ทั้งฝั่ง MPU และ MCU) เพื่อเชื่อมต่อแอปพลิเคชันของคุณเข้ากับ Router

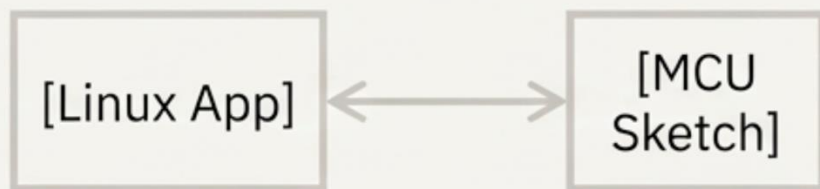
ภาพรวมสถาปัตยกรรม



Router ทำหน้าที่เป็นศูนย์กลาง (Hub) ทำให้หลายโปรเซสบน Linux และ MCU สามารถสื่อสารกันได้อย่างอิสระผ่านโครงสร้างแบบ Star Topology

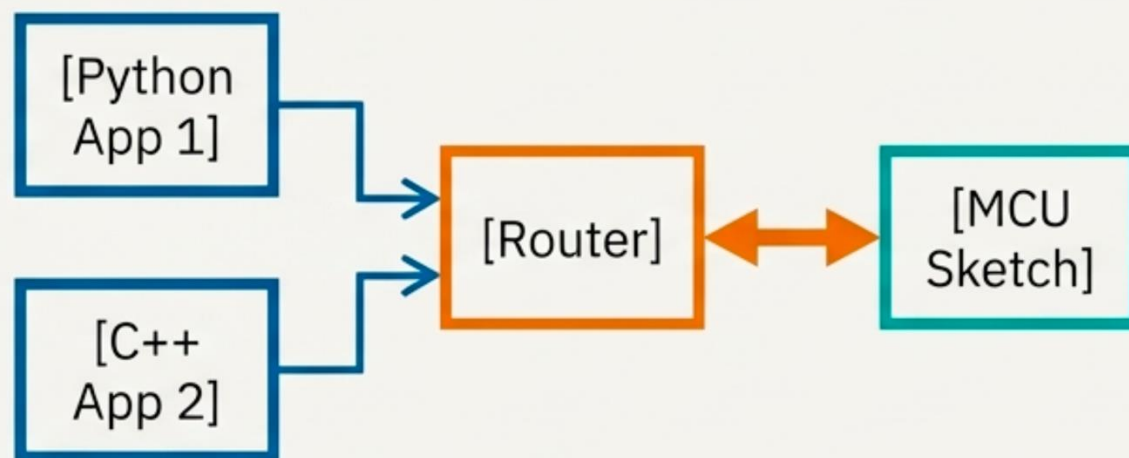
คุณสมบัติเด่น: การสื่อสารแบบหลายจุด

การสื่อสารแบบ Serial ทั้งหมด



เป็นแบบ Point-to-Point โดยมีเพียงหนึ่งโปรเซสที่สามารถคุยกับ MCU ได้ในเวลาเดียว

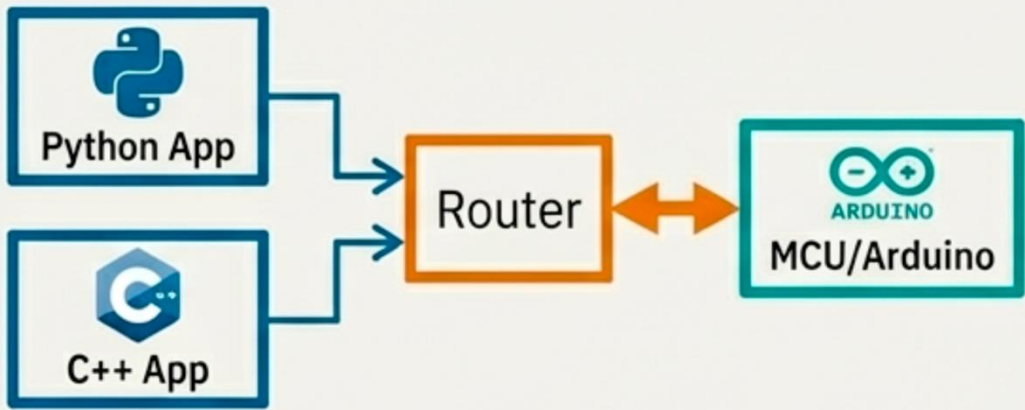
สถาปัตยกรรมผ่าน Router



Router อนุญาตให้หลายโปรเซสบน Linux ติดต่อกับ MCU และยังสามารถติดต่อกันเองได้พร้อมกัน

Multipoint Communication ในการใช้งานจริง

Linux ↔ MCU

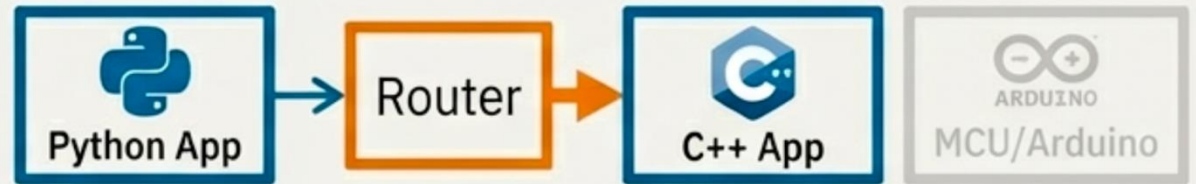


หลายโปรเซสบน Linux สามารถสื่อสารกับ MCU พร้อมกันได้

ตัวอย่าง

- โปรเซส Python® หนึ่งตัวกำลังอ่านค่าเซนเซอร์จาก MCU
- ในขณะเดียวกัน แอป C++ อีกตัวกำลังส่งคำสั่งควบคุมมอเตอร์ไปยัง MCU

Linux ↔ Linux



Router สามารถเชื่อมแอปพลิเคชันที่รันบน MPU เข้าด้วยกันได้โดยตรง

ตัวอย่าง

- Python script หนึ่งตัวทำหน้าที่เปิดเผยฟังก์ชันผ่าน RPC
- แอป Python® หรือ C++ อีกตัวสามารถเรียกใช้ฟังก์ชันนั้นได้โดยตรงผ่าน Router โดยไม่ต้องผ่าน MCU เลย

ลงมือเขียนโค้ด: สั่งเปิด-ปิด LED ผ่าน RPC

ฝั่ง Linux (Python)

```
from arduino.app_utils import *
import time

led_state = False

def loop():
    global led_state
    time.sleep(1)
    led_state = not led_state
    # Call the function on the MCU
    Bridge.call("set_led_state", led_state)

App.run(user_loop=loop)
```

ส่งคำขอเรียกฟังก์ชัน
`set\_led\_state` บน MCU

ฝั่ง MCU (Arduino C++)

```
#include "Arduino_RouterBridge.h"

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    Bridge.begin();
    // Provide the 'set_led_state' function to Linux
    Bridge.provide("set_led_state", set_led_state);
}

void loop() { }
```

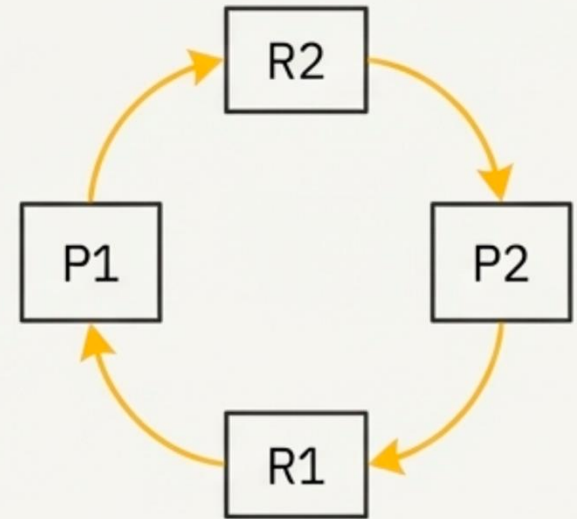
```
void set_led_state(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);
}
```

เปิดเผยฟังก์ชัน `set\_led\_state`
ให้ฝั่ง Linux สามารถเรียกใช้ได้

⚠️ ข้อควรระวัง: Threading และการเกิด Deadlock

The Problem

- ฟังก์ชันที่ถูกลงทะเบียบผ่าน `provide()` จะถูกรันใน background thread แยกต่างหาก
- `Bridge.call()` เป็น blocking call ที่จะหยุดรอนกว่าจะได้รับการตอบกลับ



The Rule

****ห้ามใช้ `Bridge.call()` (หรือ `Monitor.print()`) ภายในฟังก์ชันที่ถูกเปิดด้วย `provide()` โดยเด็ดขาด****

The Consequence

การทำเช่นนี้จะทำให้ระบบเกิด Deadlock เนื่องจาก thread ที่รอการตอบกลับ (`call`) คือ thread เดียวกันกับที่ต้องประมวลผลคำขอ ทำให้ระบบหยุดทำงาน

The Solution

ใช้ `provide_safe()` สำหรับงานที่ต้องเรียกใช้ Arduino API ทั่วไป หรือออกแบบการสื่อสารของคุณใหม่เพื่อหลีกเลี่ยงการเรียกซ้อนกัน

เครื่องมือสำหรับนักพัฒนา MCU: องค์ประกอบหลักของ Bridge

`BridgeClass`

คลาสหลักสำหรับจัดการ RPC ทั้งฝั่ง client และ server

-  **begin()**: เริ่มต้นการทำงานของ Bridge และระบบ serial ภายใน
-  **call()**: เรียกฟังก์ชันที่อยู่ฝั่ง Linux และรอผลลัพธ์กลับมา
-  **notify()**: เรียกฟังก์ชันฝั่ง Linux โดยไม่รอผลลัพธ์ (fire-and-forget)
-  **provide()**: เปิดฟังก์ชันของ MCU ให้ฝั่ง Linux เรียกใช้ (รันใน thread ที่มีความสำคัญสูง)
-  **provide_safe()**: รูปแบบที่ปลอดภัยกว่าของ `provide()` โดยจะรันฟังก์ชันใน `loop()` หลัก เหมาะสำหรับ Arduino API ส่วนใหญ่

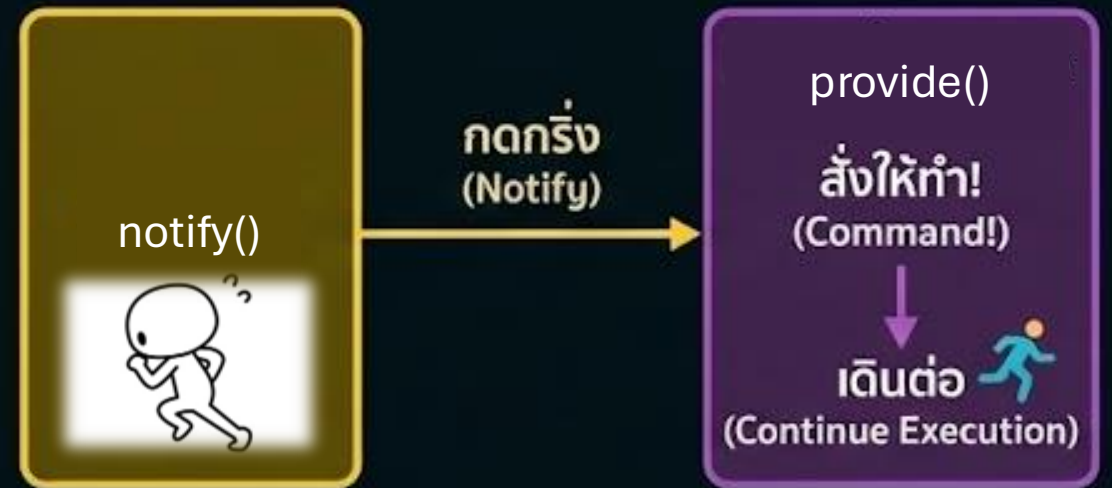
การสื่อสารของ Bridge บน Arduino UNO Q

📞 `call()` – รอคำตอบ (Wait and Return)



ถ้าต้องการ 'ค่า' → ใช้ `call()`

🔔 `notify()` – ไม่รอผล (Fire and Forget)



ถ้าแค่ 'สั่งให้ทำ' → ใช้ `notify()`

สรุปสั้นที่สุด: 🔔 `notify` = Fire and Forget | 📞 `call` = Wait and Return

RPC thread คืออะไร

RPC thread (Remote Procedure Call thread) คือ

เธรดพิเศษที่ใช้รับคำสั่งจากฝั่ง Linux (Python)
แล้วเรียกฟังก์ชันในฝั่ง MCU (C++) ทันที

พูดง่าย ๆ คือ

➡ เป็นเส้นทางด่วนจาก Python → MCU

ตำรวจจราจรคนนี้อยู่ฝั่ง Linux
แต่โบกถนนไปถึง MCU ได้

Arduino Router



provide()



Loop() Thread

RPC Thread

provide()

Python Application



Arduino API

provide_safe



ถนนคนละเส้น แต่ปลายทาง
คือ MCU เหมือนกัน

Threading & Safety

- **provide():** รับฟังก์ชัน RPC ใน Thread ที่มีความสำคัญสูง (High-priority thread) เหมาะสำหรับงานด่วนที่ต้องตอบสนองเร็ว
- **provide_safe():** ปลอดภัยกว่า โดยจะผูกฟังก์ชัน RPC ให้รับใน **loop()** หลักของ Arduino เหมาะสำหรับ Arduino API ทั่วไปที่ไม่ใช่ Thread-safe



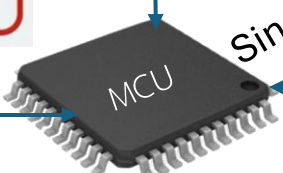
คำเตือน: Deadlock!

ห้ามใช้ **Bridge.call()** หรือ **Monitor.print()** ภายในฟังก์ชันที่ถูกบล็อกโดย **provide()**
โดยเด็ดขาด เพราะ Thread จะหยุดรอการตอบกลับจาก Linux ในขณะที่ Linux อาจกำลังรอผลจาก Thread เดียวกัน ทำให้ระบบค้าง (Deadlock)

provide()

Loop()

provide_safe()



Single / limited concurrency
Bridge

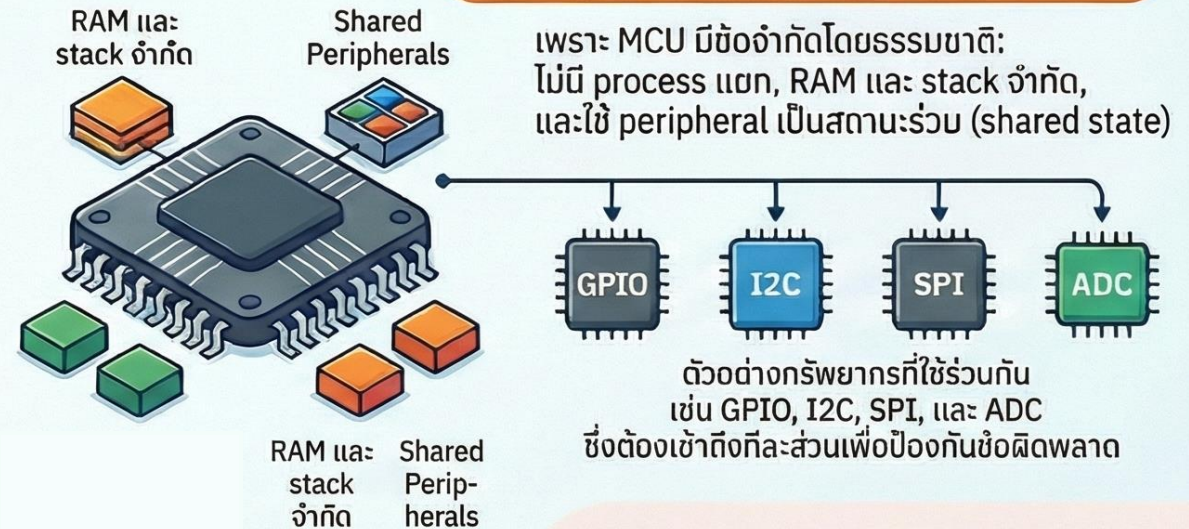
RPC thread เป็นตัวรับ RPC ทุกอัน
ถ้าใช้ **provide()** มันจะเรียก callback ใน thread ของมันเอง
แต่ถ้าใช้ **provide_safe()** มันจะฝากงานไว้ แล้วให้ **loop()** เป็นคนเรียกแทน

ไขข้อสงสัย: Single Concurrency บน MCU คืออะไร?

Single Concurrency คืออะไร?



ทำไม MCU ถึงต้องทำงานแบบนี้?



จะเกิดอะไรขึ้น? ถ้าอนุญาตให้รันพร้อมกัน



- State พัง
- Register ชนกัน
- Timing เพี้ยน

ทำให้ระบบทำงานผิดพลาด

เลือกให้ถูก! `provide()` vs. `provide_safe()` บน Arduino UNO Q

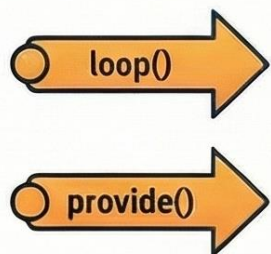
เข้าใจความเสี่ยงทางเทคนิคของ `provide()` และวิธีการที่ `provide_safe()` จัดการคำสั่งอย่างปลอดภัยผ่านระบบคิว เพื่อความเสถียรของระบบ

Section: `provide()` - ความเสี่ยงที่ต้องเลี่ยง



ขัดขวางการสื่อสาร
รับคำสั่งใน Thread สื่อสาร
ทำให้การเชื่อมต่อระหว่างชิป
ค้างหรือหลุดได้

ทำให้ท่อบัข้อมูล "ตัน"
คำสั่งที่ต้องรอ (เช่น `delay()`,
`analogRead()`) จะบล็อกระบบ
ทำให้เกิด Timeout Error



เกิด Race Condition
การเรียก API พร้อมกันจาก
`'loop()'` และ `'provide()'`
อาจทำให้ข้อมูลขัดแย้งกัน

Section: `provide_safe()` - กลไกเพื่อความปลอดภัย

ทำงานด้วยหลักการ "ฝากข้อความ" ผ่านคิว

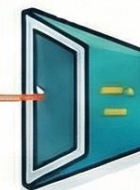
ไม่รับคำสั่งทันที แต่จัดคิวรอจังหวะที่ปลอดภัย เพื่อความเสถียร 100%



Linux



Command

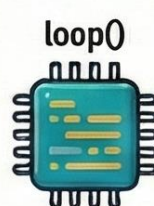


1. รับคำสั่ง

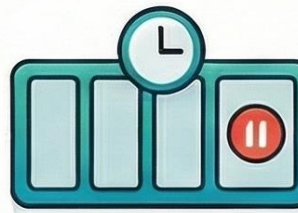
เมื่อ Linux ส่งคำสั่งมา
`'provide_safe()'` จะรับไว้ก่อน

2. เก็บเข้าคิว

นำคำสั่งไปเก็บไว้ในคิวพิเศษ
แทนที่จะรันทันที



loop0



3. รอจังหวะปลอดภัย

ระบบจะรอนกว่า `'loop0()'` หลัก
ว่าง หรือถึงจังหวะที่เหมาะสม



Main Thread

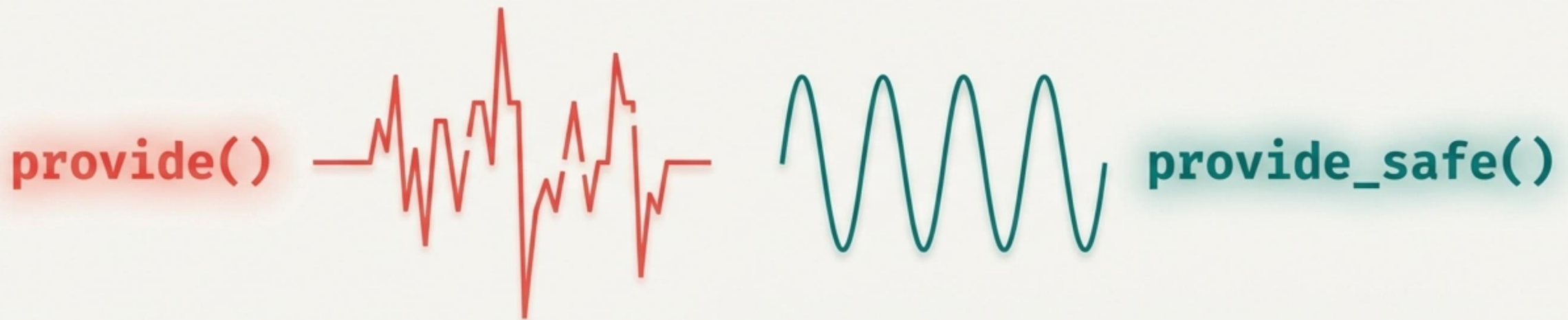


4. ถึงมาทำงาน

ดึงคำสั่งจากคิวมาทำงานใน
Main Thread ที่ปลอดภัย

โค้ด Arduino UNO Q ของคุณแข็งแกร่งไหม? สาเหตุอาจอยู่ที่ **provide()**

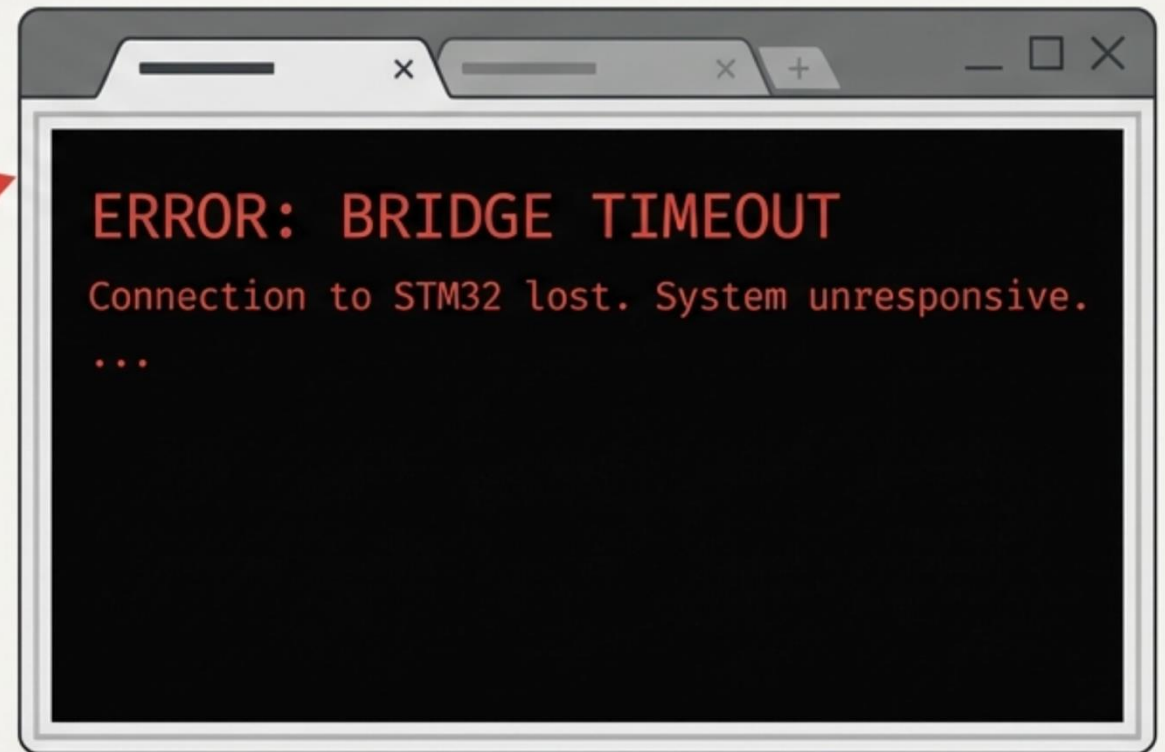
ทำความเข้าใจความแตกต่างที่สำคัญระหว่าง **provide()** และ **provide_safe()** เพื่อสร้างระบบที่เสถียรและเชื่อถือได้



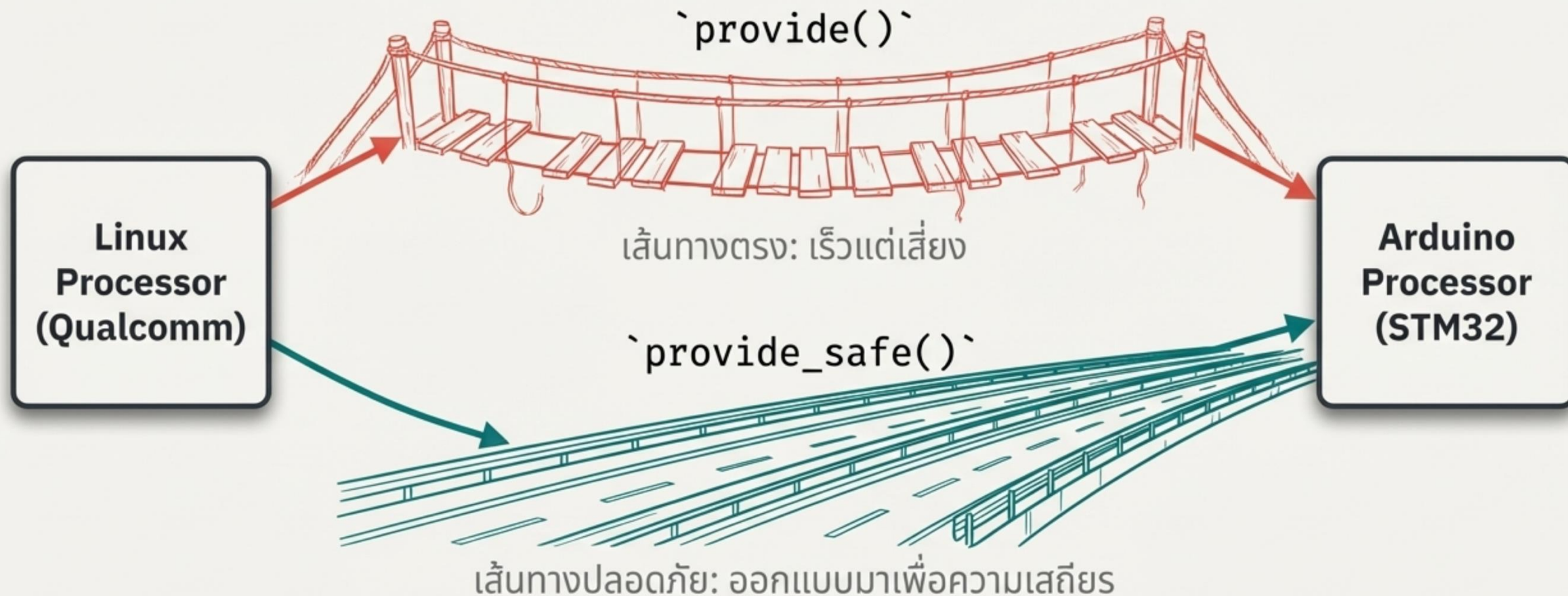
สถานการณ์ที่คุ้นเคย: การเรียกใช้ API ง่ายๆ ที่ทำให้ทั้งระบบหยุดทำงาน

คุณต้องการควบคุมฮาร์ดแวร์จากฝั่ง Linux (Python) ผ่าน Bridge Callback มันดูเหมือนจะตรงไปตรงมา แต่ผลลัพธ์ที่ได้คือระบบค้าง, การเชื่อมต่อหลุด, หรือเกิด Timeout Error ที่ไม่สามารถคาดเดาได้

```
void myCallback(String& value) {  
    digitalWrite(13, HIGH);  
    delay(500); // <— The problem  
    digitalWrite(13, LOW);  
}  
  
void setup() {  
    Bridge.provide("led", myCallback);  
    // ...  
}
```



การสื่อสารข้าม Bridge: สองเส้นทาง สองผลลัพธ์



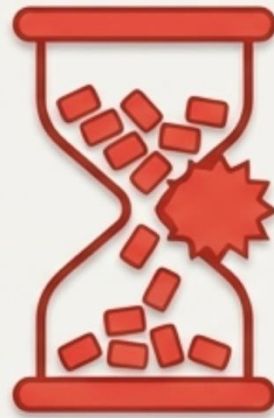
ทั้งสองฟังก์ชันมีเป้าหมายเดียวกันคือส่งคำสั่งข้ามจาก Linux ไปยัง Arduino แต่ ‘วิธีการ’ ที่มันทำงานนั้นแตกต่างกันอย่างสิ้นเชิง และนี่คือจุดที่กำหนดความเสถียรของระบบคุณ

เปิดโปง 3 อันตรายที่ซ่อนอยู่ในการใช้ `provide()` กับ Arduino API

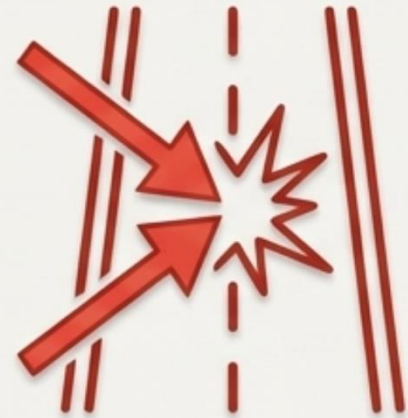
การที่ระบบของคุณล่มไม่ได้เกิดจากความผิดพลาดแบบสุ่ม แต่มีเหตุผลทางเทคนิคที่ชัดเจน 3 ประการที่ทำให้การเรียกใช้ `digitalWrite`, `analogRead`, หรือ `delay` ใน `provide()` เป็นความคิดที่อันตราย



1. ความขัดแย้งของ Context
(บริบทการทำงาน)



2. กั๊กของ
Blocking & Timing

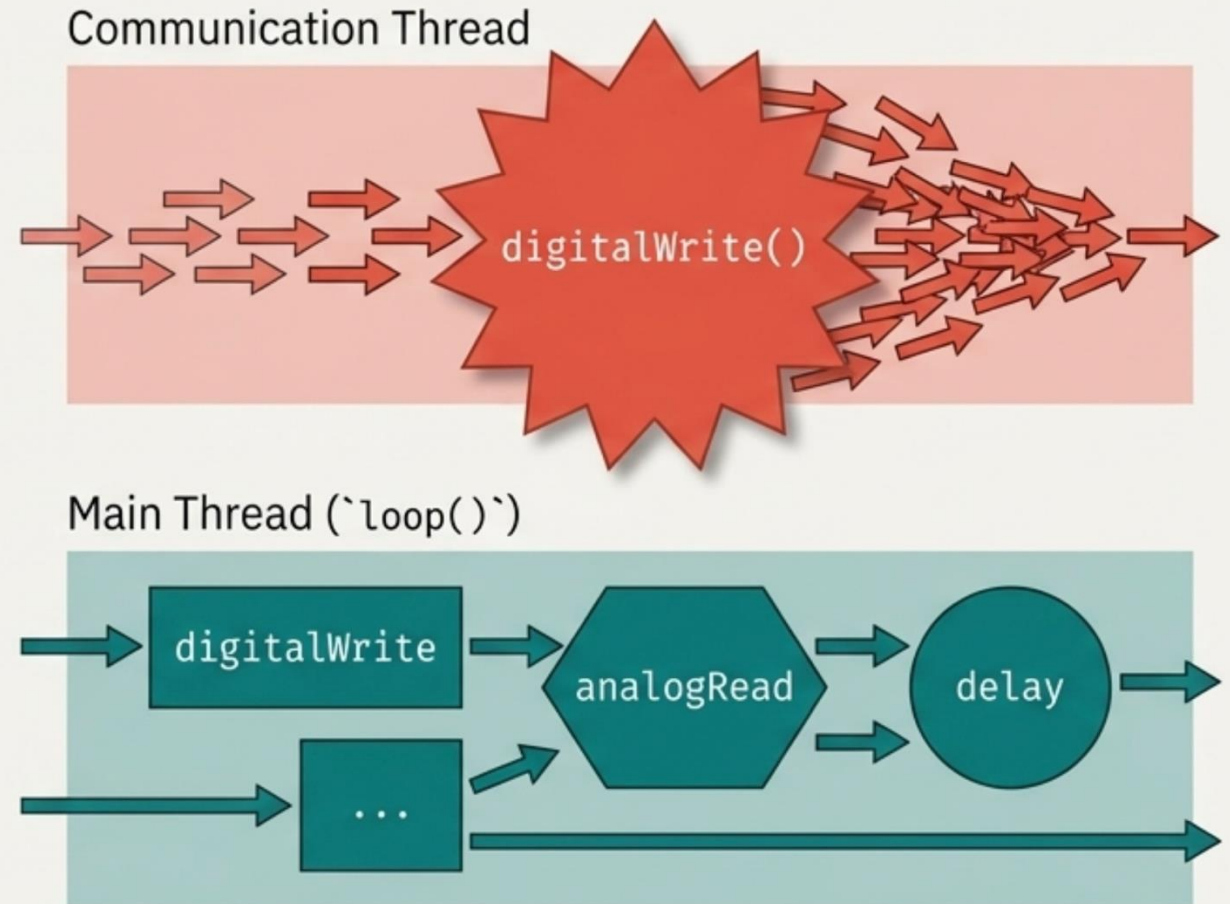


3. การแข่งขันของ Thread
(Race Condition)

อันตรายที่ 1: ความขัดแย้งของ Context

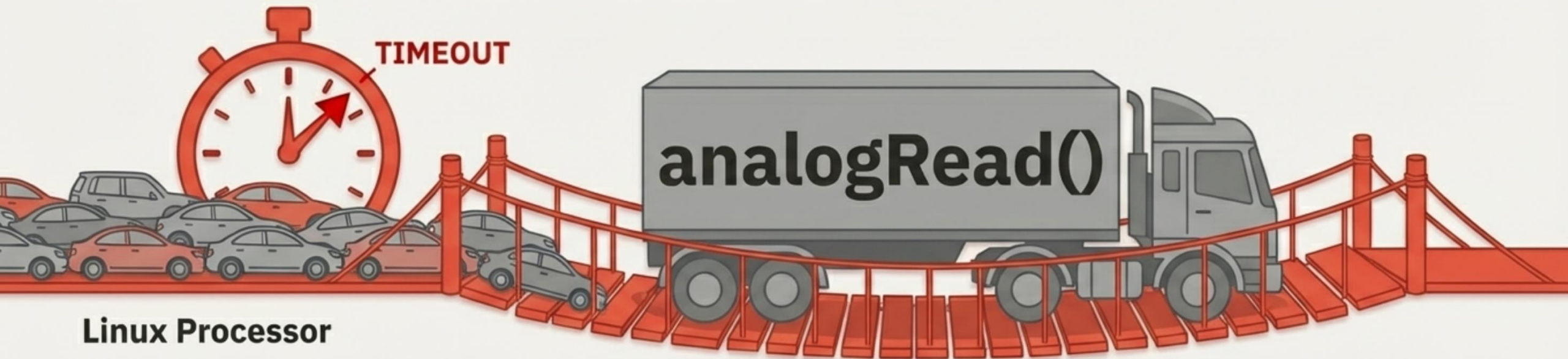
`provide()` ทำงานในบริบทที่ต่างออกไป

- **Bridge.provide():** Callback ของคุณจะถูก
รันใน **Communication Thread** โดยตรง
ซึ่งเป็น 'เลนด่วน' ที่สงวนไว้สำหรับการรับส่ง
ข้อมูล (Handshake) ระหว่างสองโปรเซสเซอร์
เท่านั้น
- **Main loop():** เป็นบริบทหลักที่ Arduino API
API ถูกออกแบบมาให้ทำงานอย่างปลอดภัย
- **ปัญหา:** การเรียกใช้คำสั่งฮาร์ดแวร์ใน
Communication Thread ก็เหมือนกับการ
'จอดรถซ่อมกลางทางด่วน' มันขัดขวางการ
สื่อสารที่สำคัญ ทำให้ Bridge หยุดชะงักและ
ระบบทั้งหมด 'ค้าง'



อันตรายที่ 2: กับดักของ Blocking และ Timing

Arduino API คลายตัว ‘หยุดรอ’ เพื่อทำงานให้เสร็จ

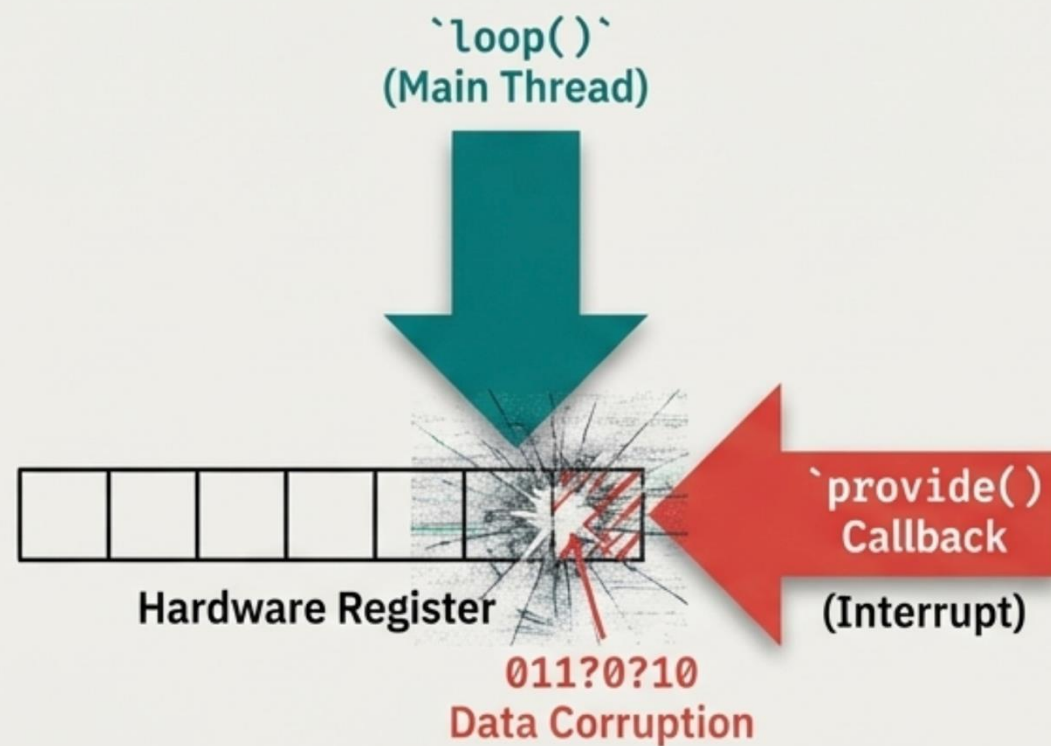


- คำสั่งเช่น ``delay()`, `analogRead()`, และแม้กระทั่ง `digitalWrite()`, เป็นแบบ Blocking—มันจะหยุดการทำงานของ Thread ทั้งหมดไว้จนกว่าจะเสร็จสิ้น`
- เมื่อคำสั่งเหล่านี้ถูกเรียกใน ``provide()`, มันจะทำให้ท่อส่งข้อมูลทั้งหมด ‘ตัน’ ชั่วขณะ`
- ฝั่ง Linux (Python) ที่รอการตอบสนอง จะไม่ได้รับการติดต่อกลับ และตัดสินใจเกิด **Timeout Error**

อันตรายที่ 3: การแข่งขันของ Thread (Race Condition)

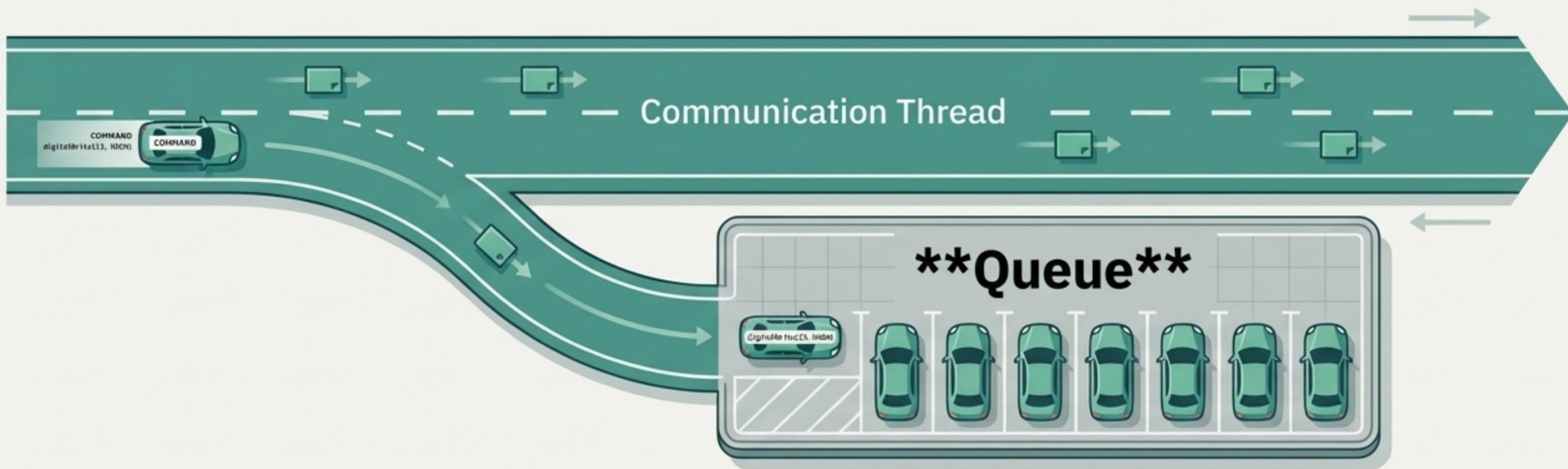
Arduino API ส่วนใหญ่ไม่เป็น Thread-safe

- **Thread-safe** หมายถึงการที่โค้ดสามารถถูกเรียกใช้จากหลาย Thread พร้อมกันได้โดยไม่เกิดปัญหา
- Arduino API ส่วนใหญ่ **ไม่ถูกออกแบบ** มาให้เป็น Thread-safe
- **ปัญหา:** ลองนึกภาพว่า `loop()` กำลังเขียนข้อมูลลงในรีจิสเตอร์ของฮาร์ดแวร์ แล้วจู่ๆ `provide()` (ที่ทำงานแทรกขึ้นมาใน Interrupt) ก็พยายามเขียนข้อมูลลงในรีจิสเตอร์เดียวกันพร้อมกัน ผลลัพธ์คือข้อมูลเพี้ยน (Data Corruption) และการทำงานที่ผิดพลาดอย่างคาดเดาไม่ได้



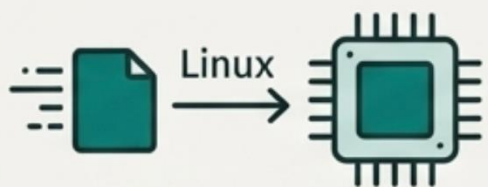
ทางออก: `provide_safe()` และหลักการ 'ฝากข้อความ'

ออกแบบมาโดยเฉพาะเพื่อแก้ปัญหาทั้ง 3 ข้อ ด้วยสถาปัตยกรรมที่ชาญฉลาด

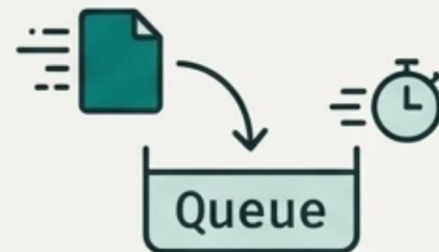


แทนที่จะรันคำสั่งทันทีในบริบทที่อันตราย `provide_safe()` จะนำคำสั่งของคุณไป 'เข้าคิว' (Queueing) และรอจังหวะที่ปลอดภัยที่สุดเพื่อทำงาน

เจาะลึกการทำงานของ `provide\_safe()`



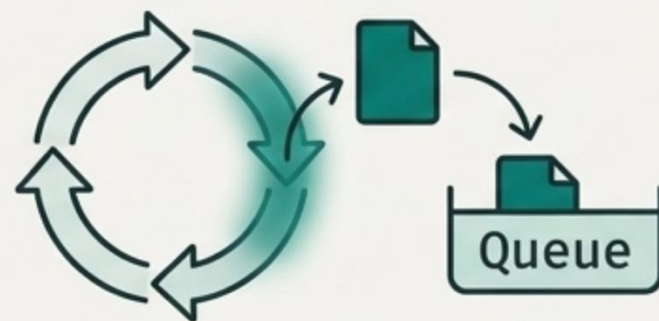
1. คำสั่งจาก Linux มาถึง



2. `provide\_safe()` รับคำสั่งและนำไปเก็บไว้ในคิวพิเศษทันที โดยไม่รบกวน Communication Thread



3. ระบบรอจนถึงจังหวะที่ `loop()` ทำงานว่างหรืออยู่ในสถานะที่ปลอดภัยต่อฮาร์ดแวร์



4. ระบบดึงคำสั่งออกจากคิวและรันใน Main Thread ซึ่งเป็นบริบทที่ถูกต้องและปลอดภัย 100%

สรุปเปรียบเทียบ: `provide()` vs. `provide\_safe()`

	<code>provide()</code>	<code>provide_safe()</code>
บริบทการทำงาน (Context)	✗ ทำงานใน Communication Thread (เสี่ยงต่อการ Hang)	✓ ทำงานใน Main Thread (ปลอดภัย)
การจัดการ Blocking Call	✗ บล็อกการสื่อสารทั้งหมด (เสี่ยงต่อ Timeout)	✓ ไม่บล็อกการสื่อสาร (รับแบบ Asynchronous)
ความปลอดภัยของ Thread	✗ ไม่ Thread-safe (เสี่ยงต่อ Race Condition)	✓ ปลอดภัยโดยสถาปัตยกรรม (ทำงานใน Thread เดียว)

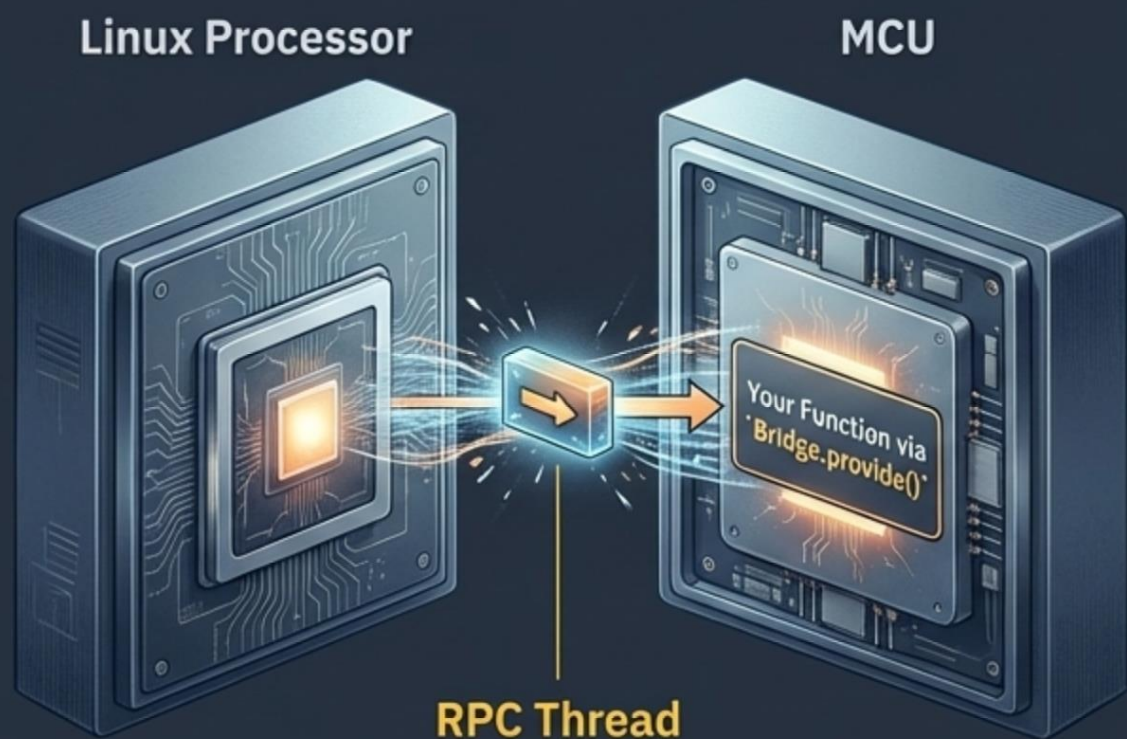
The Golden Rule: กฎเหล็กเพื่อโค้ดที่เสถียร

“เมื่อใดก็ตามที่ Callback ของคุณต้องยุ่งเกี่ยวกับฮาร์ดแวร์—ไม่ว่าจะเป็น digitalWrite, analogRead, delay หรือ API อื่นๆ—**จงใช้ `provide_safe()` เสมอ***”



การเลือกใช้เครื่องมือที่ถูกต้องไม่เพียงแค่ป้องกันบั๊ก แต่คือหัวใจของการเขียนโค้ดระดับมืออาชีพที่ทั้งทนทาน, คาดเดาได้, และง่ายต่อการบำรุงรักษา

⚠ ขอแนะนำ `Bridge.provide()`: พื้นที่อันตรายที่ทุกคำสั่งต้องระวัง



- อธิบายสั้นๆ ว่าฟังก์ชันที่ถูกเรียกผ่าน `Bridge.provide()` ทำงานใน 'RPC thread' (Remote Procedure Call thread)
- เน้นย้ำว่า thread นี้มีความสำคัญอย่างยิ่งต่อการสื่อสารระหว่างโปรเซสเซอร์
- การกระทำใดๆ ที่ขัดขวาง thread นี้ จะส่ง **ผลร้ายแรง** ต่อเสถียรภาพของทั้งระบบ

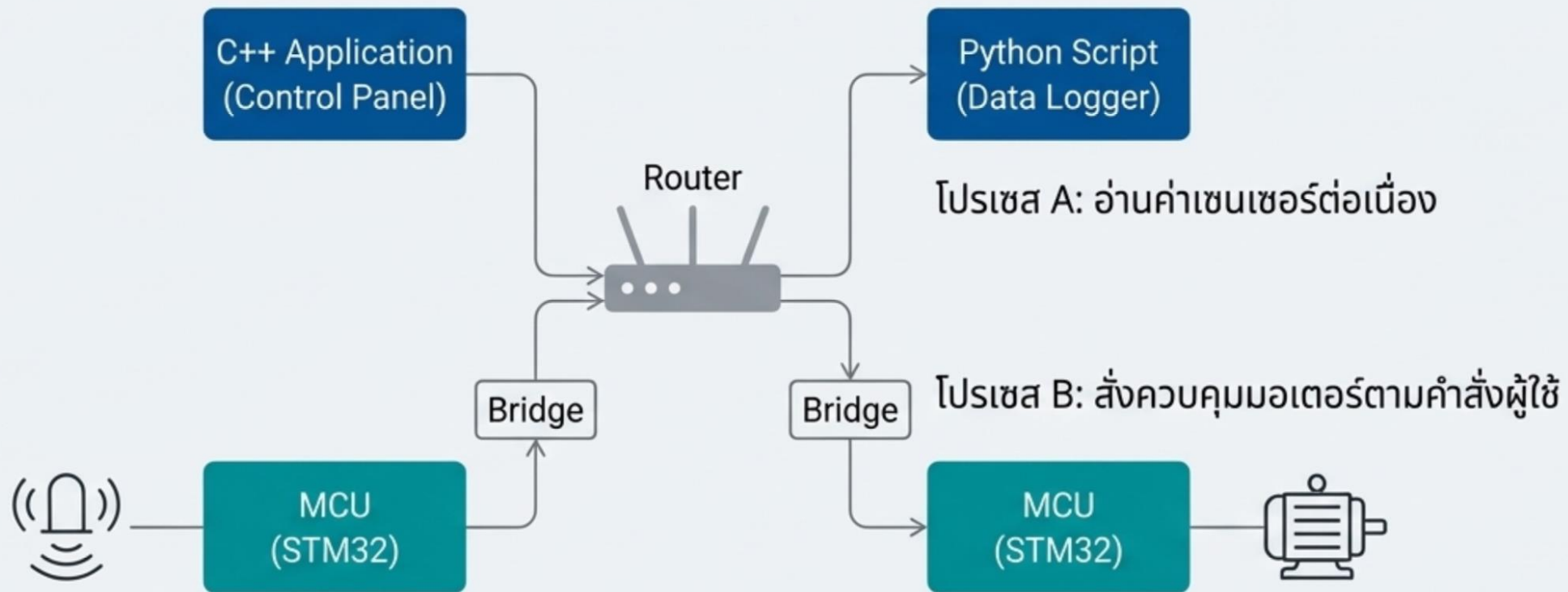
แม้คุณจะเขียนโค้ดโดยใช้ **Bridge** library แต่หัวใจของระบบสื่อสารทั้งหมดคือ Service บน Linux ที่ชื่อว่า **arduino-router** มันคือผู้ควบคุมเส้นทางการจราจรข้อมูล (Traffic Controller) ทั้งหมดที่ทำงานอยู่เบื้องหลัง



Router ไม่ใช่แค่ท่อส่งข้อมูลแบบ 1-ต่อ-1 แต่เป็นศูนย์กลางการสื่อสารอัจฉริยะที่เชื่อมต่อทุกส่วนเข้าด้วยกัน

กรณีศึกษา 1: การสื่อสารระหว่าง Linux ↔ MCU

ควบคุมและเฝ้าดูสถานะของ MCU พร้อมกันจากโปรแกรมที่ต่างกัน

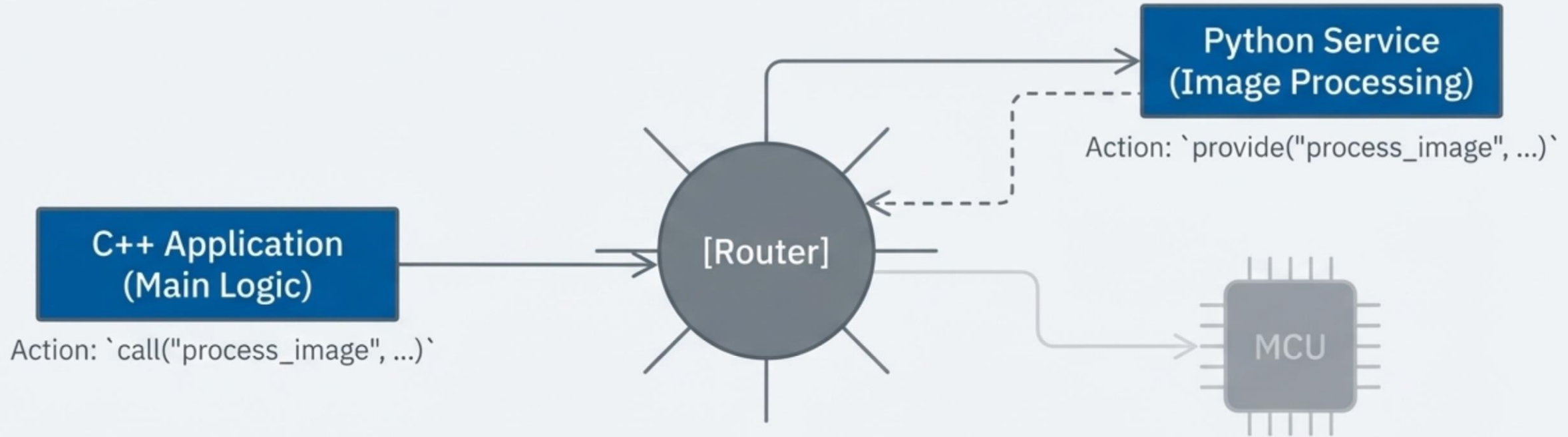


- โปรเซสหนึ่งสามารถร้องขอข้อมูล (Sensor Reading) จาก MCU ได้
- ในขณะเดียวกัน อีกโปรเซสหนึ่งสามารถส่งคำสั่ง (Motor Control) ไปยัง MCU ได้
- 'arduino-router' รับหน้าที่จัดการส่งต่อข้อความไปยังปลายทางที่ถูกต้อง ทำให้ทั้งสองโปรเซสทำงานพร้อมกันได้โดยไม่รบกวนกัน

กรณีศึกษา 2: การสื่อสารระหว่าง Linux ↔ Linux

สร้างสถาปัตยกรรมแบบ Microservices บนบอร์ด

Router ไม่ได้จำกัดแค่การคุยกับ MCU แต่ยังสามารถใช้เป็น Bus สำหรับการสื่อสารระหว่างโปรเซส (IPC) บน Linux ได้ด้วย



Key Takeaway: คุณสามารถสร้างแอปพลิเคชันแบบโมดูลาร์ โดยให้โปรเซสหนึ่งทำหน้าที่เป็น 'Service' ที่เปิดฟังก์ชัน RPC ของตัวเอง และให้โปรเซสอื่นมาเรียกใช้งานได้

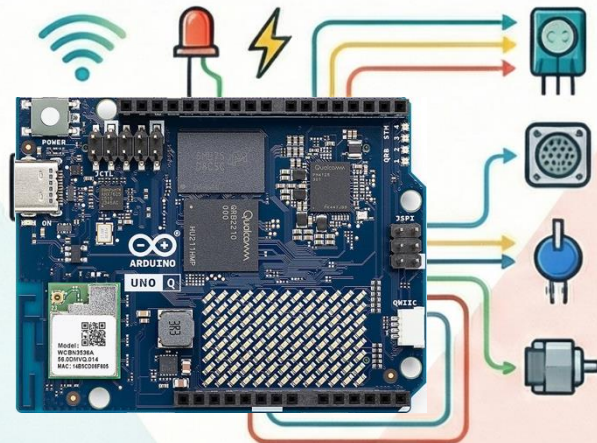
เจาะลึก Arduino API: คำสั่งพื้นฐานและข้อควรระวัง

ทำความรู้จัก Arduino API



Arduino API คืออะไร?

ชุดฟังก์ชันมาตรฐานที่ Arduino เตรียมไว้ให้เราใช้ควบคุมฮาร์ดแวร์ได้ง่ายขึ้น



ตัวอย่างและข้อควรระวัง

```
sample {  
  pinMode()  
  digitalWrite()  
  delay()  
  analogRead()  
}
```

pinMode()
digitalWrite()
delay()
analogRead()



เป็นภาษากลางในการเขียนโค้ด

ทำให้เราสามารถเขียนโค้ดควบคุมบอร์ด Arduino รุ่นไหนก็ได้ในรูปแบบเดียวกัน

ประเภทคำสั่งหลัก



อ่าน/เขียนขา



จัดการเวลา



สื่อสาร



คำนวณ



ควบคุมฮาร์ดแวร์



คำสั่งที่ "ไม่เหมาะสมเกินขนาด" ใน Bridge.provide()
การใช้คำสั่งเหล่านี้อาจทำให้เกิดปัญหา deadlock, block, หรือ starvation

	คำสั่ง (Command)	เหตุผล (Reason)
	`delay()`	block RPC thread
	`Bridge.call()`	recursive deadlock
	งาน loop ยาว	starvation

ชุดคำสั่งพื้นฐานที่เราใช้กันทุกวัน



ควบคุมขา I/O

```
pinMode(13, OUTPUT);  
digitalWrite(13, HIGH);  
digitalRead(2);
```



จัดการเวลา

```
delay(1000);  
millis();
```



สื่อสาร (Serial)

```
Monitor.begin();  
Monitor.println("Hello");
```



สัญญาณ Analog

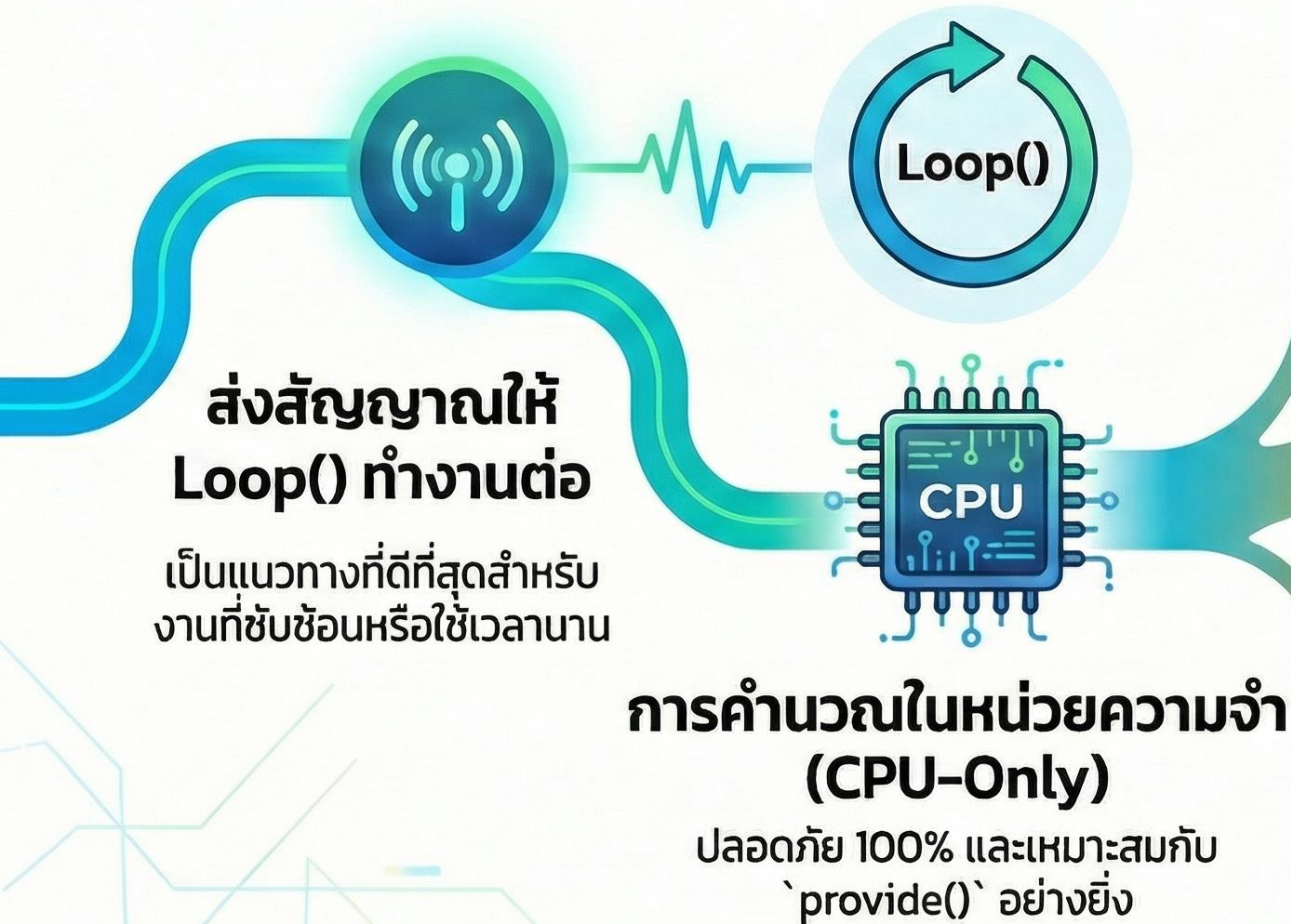
```
analogRead(A0);  
analogWrite(9, 128);
```

บัญชีดำ: สรุปคำสั่งที่ 'ไม่เหมาะสมเมื่อดัดขาด' ใน `Bridge.provide()`

คำสั่ง	เหตุผล
<code>delay()</code>	block RPC thread
<code>Serial.print()</code>	ใช้ mutex
<code>Monitor.print()</code>	deadlock
<code>Bridge.call()</code>	recursive deadlock
<code>Bridge.notify()</code>	mutex ช้อน
<code>analogRead()</code>	ช้า / block
<code>Wire.*</code>	I2C block
<code>SPI.*</code>	block
งาน loop ยาว	starvation

แนวทางการเขียน Callback สำหรับ Bridge.provide()

รูปแบบที่แนะนำที่สุด (Most Recommended Patterns)



รูปแบบอื่นที่ใช้ได้ (Other Usable Patterns)



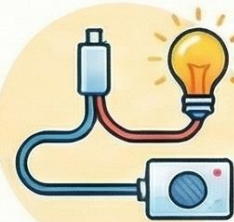
การเปลี่ยนค่า State / Flag

เป็นวิธีที่ใช้บ่อยและปลอดภัย สามารถทำงานร่วมกับ `loop()` ได้ดี



การอัปเดตตัวแปร Global / Struct

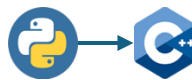
ปลอดภัย トラบใดที่ไม่มีการเรียกใช้ I/O



งาน GPIO ที่ “เร็วมาก”

⚠ ใช้ได้ แต่มีเงื่อนไขคือต้องไม่มี `delay`, ไม่มี `Serial` และต้องสั้นมาก

Bridge.provide()



1 การคำนวณในหน่วยความจำ (CPU-only)

```
cpp
int a = 1, b = 2, c;

void sum() {
    c = a + b;
}
```

- ✓ ปลอดภัย 100%
- ✓ เหมาะมากสำหรับ `provide()`

2 การเปลี่ยนค่า state / flag

```
cpp
bool flag = false;

void set_flag() {
    flag = true;
}
```

- ✓ ใช้บ่อย
- ✓ ใช้ร่วมกับ `loop()` ได้ดี

3 การอัปเดตตัวแปร global / struct

```
cpp
struct Data {
    int x;
    int y;
};

Data d;

void update_data() {
    d.x++;
    d.y = 10;
}
```

- ✓ ปลอดภัย
- ✓ ไม่มี I/O

5 การตั้งค่าเพื่อให้ `loop()` ไปทำต่อ

```
cpp
volatile bool do_work = false;

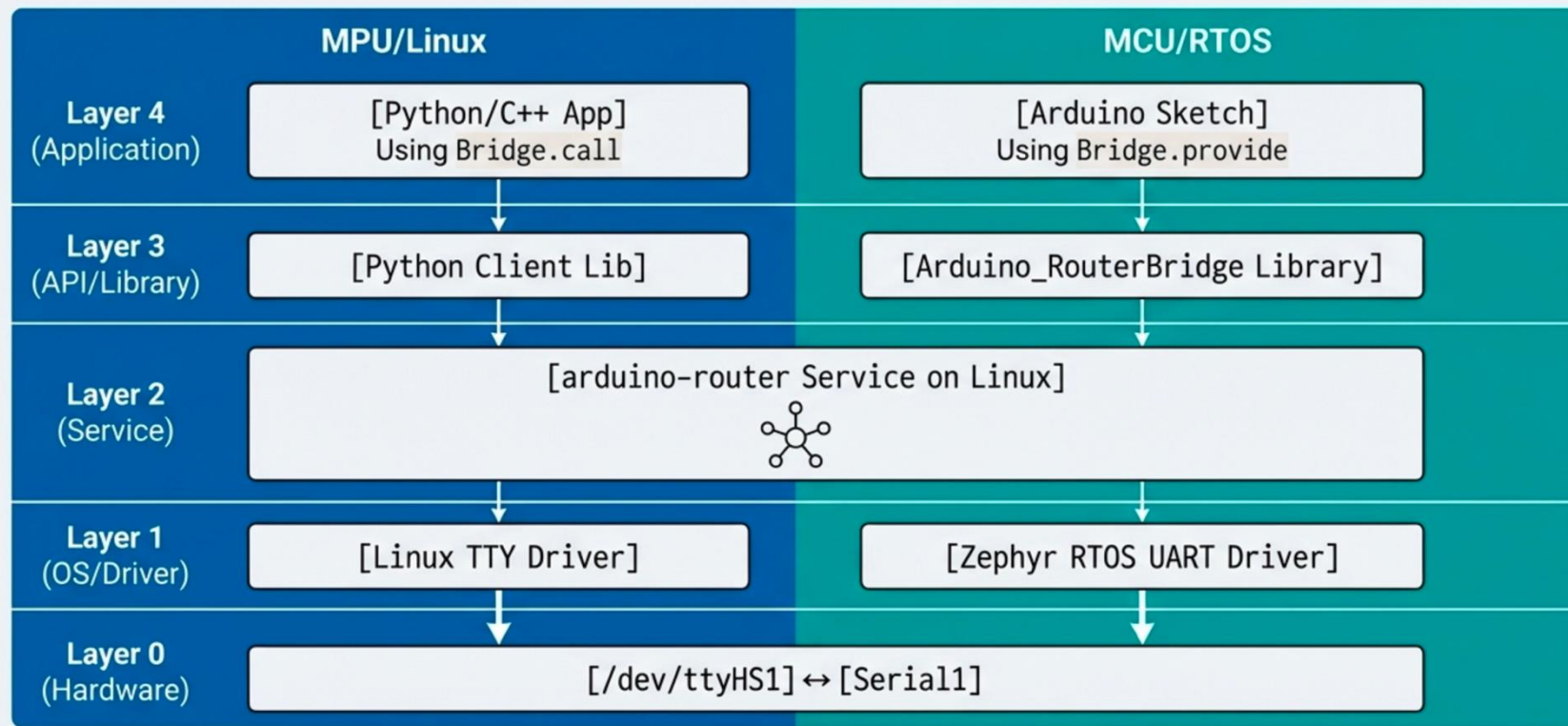
void request_work() {
    do_work = true;
}

void loop() {
    if (do_work) {
        do_work = false;
        // งานยาว ๆ ทำที่นี่
    }
}
```

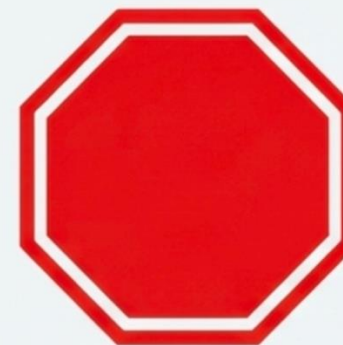
- ✓ แนวทางที่ "แนะนำที่สุด" สำหรับงานจริง

`arduino-router` เป็นผู้จัดการการเชื่อมต่อทางกายภาพระหว่าง
โปรเซสเซอร์ทั้งสองผ่านพอร์ตอนุกรมความเร็วสูง (High-Speed UART)

ภาพรวมสถาปัตยกรรมทั้งหมด: จากโค้ดสู่ฮาร์ดแวร์



นี่คือภาพรวมทั้งหมดของสถาปัตยกรรมที่คุณใช้งานอยู่ ตั้งแต่ API ที่คุณเรียกในโค้ด ไปจนถึงการเชื่อมต่อทางกายภาพที่ทำให้ทุกอย่างทำงานเป็นไปได้อีก



คำเตือน: ทรัพยากรที่ถูกจองไว้

ห้ามเปิดใช้งานหรือยุ่งเกี่ยวกับ
`/dev/ttyHS1` (บน Linux) หรือ
`Serial1` (ใน Arduino Sketch)
ในโค้ดของคุณโดยตรง
อินเตอร์เฟซเหล่านี้ถูกบล็อกไว้สำหรับ
`arduino-router` เท่านั้น การพยายาม
เข้าถึงจะทำให้ระบบสื่อสารทั้งหมดล้มเหลว

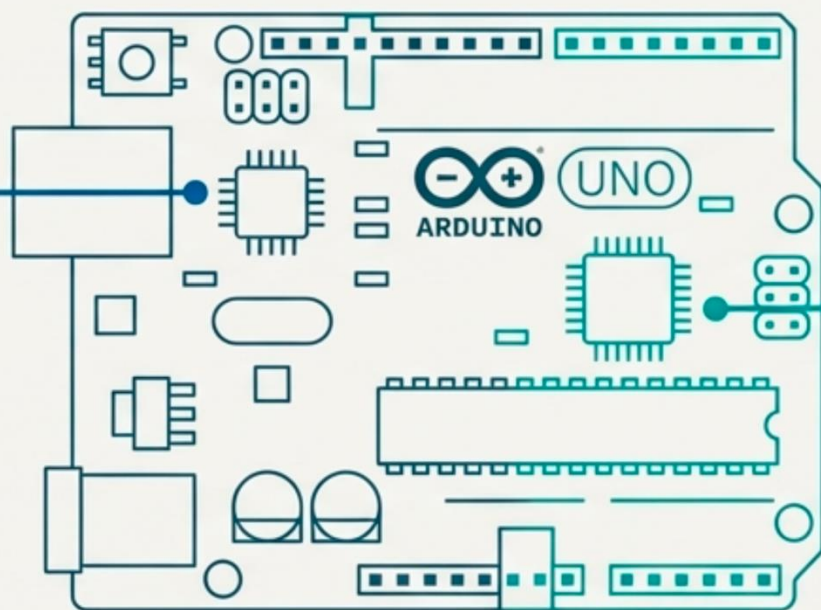
กฎการใช้งานระบบ: ทรัพยากรฮาร์ดแวร์ที่ถูกจองไว้

Arduino Router เป็นผู้จัดการการเชื่อมต่อทางกายภาพระหว่างโปรเซสเซอร์ทั้งสอง
คุณจึงต้องหลีกเลี่ยงการใช้งานพอร์ตที่ Router จองไว้

ฝั่ง Linux (MPU)

Port: `/dev/ttyHS1`

Status:  Locked by
``arduino-router``



ฝั่ง MCU (STM32)

Port: ``Serial1``

Status:  Locked by
``Arduino_RouterBridge``

! คำเตือน: ห้ามเปิดใช้งานหรือพยายามเข้าถึง `/dev/ttyHS1`` หรือ ``Serial1``
ในโค้ดของคุณโดยเด็ดขาด หากมีการเข้าถึงโดยตรง Bridge จะทำงานล้มเหลว

สำหรับขั้นสูง: เชื่อมต่อโดยตรงผ่าน Unix Socket

แอปพลิเคชันบน Linux ไม่ได้เชื่อมต่อกับ `Bridge` โดยตรง แต่สื่อสารกับ `arduino-router` ผ่าน Unix Domain Socket ซึ่งเป็นมาตรฐานการสื่อสารระหว่างโปรเซสบน Linux

- **Socket Path**: `/var/run/arduino-router.sock`
- **Protocol**: MessagePack RPC (a binary serialization format, efficient and fast)
- **Implication** คุณสามารถเขียน Client ของคุณเองในภาษาใดก็ได้ (Python, C++, Go, Rust) เพื่อเชื่อมต่อกับ Router โดยตรง โดยไม่จำเป็นต้องใช้ไลบรารีที่ให้มีมา

```
# ตัวอย่างการเรียก RPC โดยตรงด้วย msgpack
import msgpackrpc

# เชื่อมต่อไปยัง socket ของ router
client = msgpackrpc.Client(
    msgpackrpc.Address("/var/run/arduino-router.sock")
)

# เรียกฟังก์ชัน RPC เหมือนปกติ
result = client.call('set_led_state', True)
```

เครื่องมือสำหรับผู้ดูแล: การจัดการ Router Service

โดยปกติ `arduino-router` จะรันอัตโนมัติเป็น System Service แต่ในกรณีที่ต้องการดีบั๊กหรือรีสตาร์ทระบบสื่อสาร คุณสามารถจัดการได้ผ่าน Command Line บน Linux

****ตรวจสอบสถานะปัจจุบันของ Service:****

```
$ systemctl status arduino-router
```

● arduino-router.service - Arduino Router Service

Loaded: loaded (/etc/systemd/system/arduino-router.service; enabled; vendor preset: enabled)

Active: active (running) since Mon 2023-10-23 10:00:00 UTC; 2h 38min ago

Main PID: 1234 (arduino-router)

Tasks: 4 (limit: 4915)

Memory: 12.5M

CGroup: /system.slice/arduino-router.service

└─1234 /usr/bin/arduino-router)

****บังคับให้ Service รีสตาร์ท:****

```
$ sudo systemctl restart arduino-router
```

****ดู Log การทำงานแบบเรียลไทม์ (สำคัญมากสำหรับการดีบั๊ก):****

```
$ journalctl -u arduino-router -f
```

Oct 23 12:30:00 linux-device arduino-router[1234]: INFO: Service started successfully.

Oct 23 12:30:01 linux-device arduino-router[1234]: INFO: Connected to MCU on /dev/ttyHS1.

Oct 23 12:30:05 linux-device arduino-router[1234]: DEBUG: Received message from MCU: {"type":"data","payload":"..."}

Oct 23 12:30:10 linux-device arduino-router[1234]: INFO: Forwarding data to application.

คุณมีเครื่องมือในการตรวจสอบและแก้ไขปัญหาระบบสื่อสารอยู่ในมือ

การจัดการ Service: ควบคุมและดีบั๊ก Arduino Router

โดยปกติ `arduino-router` จะทำงานอัตโนมัติ แต่คุณสามารถจัดการได้โดยตรงผ่าน command line บน Linux

ตรวจสอบสถานะ (Check Status)

```
systemctl status arduino-router
```

รีสตาร์ทบริการ (Restart the Service)

```
sudo systemctl restart arduino-router
```

ดู Log แบบเรียลไทม์ (View Real-time Logs)

```
journalctl -u arduino-router -f
```



Pro-Tip

การเปิดโหมด Verbose สำหรับดีบั๊ก

1. แก้ไขไฟล์ service: `sudo nano /etc/systemd/system/arduino-router.service`
2. เพิ่ม flag `--verbose` ต่อท้าย `ExecStart`
3. Reload & Restart: `sudo systemctl daemon-reload && sudo systemctl restart arduino-router`

สำหรับผู้ใช้งานขั้นสูง: การเชื่อมต่อผ่าน Unix Socket โดยตรง

Linux แอปพลิเคชันไม่ได้ถูกจำกัดให้ใช้แค่ Bridge library คุณสามารถสื่อสารกับ `arduino-router` ได้โดยตรงผ่าน Unix Domain Socket

`/var/run/arduino-router.sock`

MessagePack RPC

วิธีการทำงาน

คุณสามารถเชื่อมต่อและส่งคำขอ RPC ได้จากภาษาโปรแกรมใดก็ได้ที่รองรับ Unix Sockets และ MessagePack

เช่น:
Python, C++, Rust, Go

conceptual Python

```
# Example using msgpack to call RPC
directly
import msgpackrpc

client = msgpackrpc.Client(msgpackrpc.
    Address("/var/run/arduino-router.sock"))
result = client.call('set_led_state', True)
```

นี่คือวิธีที่ทรงพลังในการผนวกรวมระบบของคุณเข้ากับแอปพลิเคชันที่หลากหลาย

กติกการใช้งาน Bridge



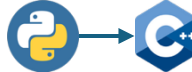
1	<code>from arduino.app_utils import Bridge, App</code>
2	ในกรณีที่ฟังก์ชันใช้งานเมธอด <code>provide()</code> หรือ <code>provide_safe()</code> จะต้องทำการสร้างออปเจ็คขึ้นมา รองรับจากคลาส <code>Bridge</code> ด้วย เช่น <code>bridge = Bridge()</code>
3	ในบรรทัดสุดท้ายจำเป็นต้องมีคำสั่ง <code>App.run()</code>



1	<code>#include "Arduino_RouterBridge.h"</code>
2	ในส่วนของฟังก์ชัน <code>setup()</code> จะต้องเริ่มต้นการใช้งานด้วยคำสั่ง <code>Bridge.begin()</code>

ชื่อ RPC service จะต้องเป็นชื่อเดียวกันทั้งสองฝั่ง

call() and provide()



โยนค่า 1 ค่า

$a = 5$



$5 + 1$



แสดงผล 1 วินาที

โยนค่ามากกว่า 1 ค่า

$a = 5, b = 7$



$5 + 7$



แสดงผล 1 วินาที

return

$a = 5, b = 7$



$5 + 7$

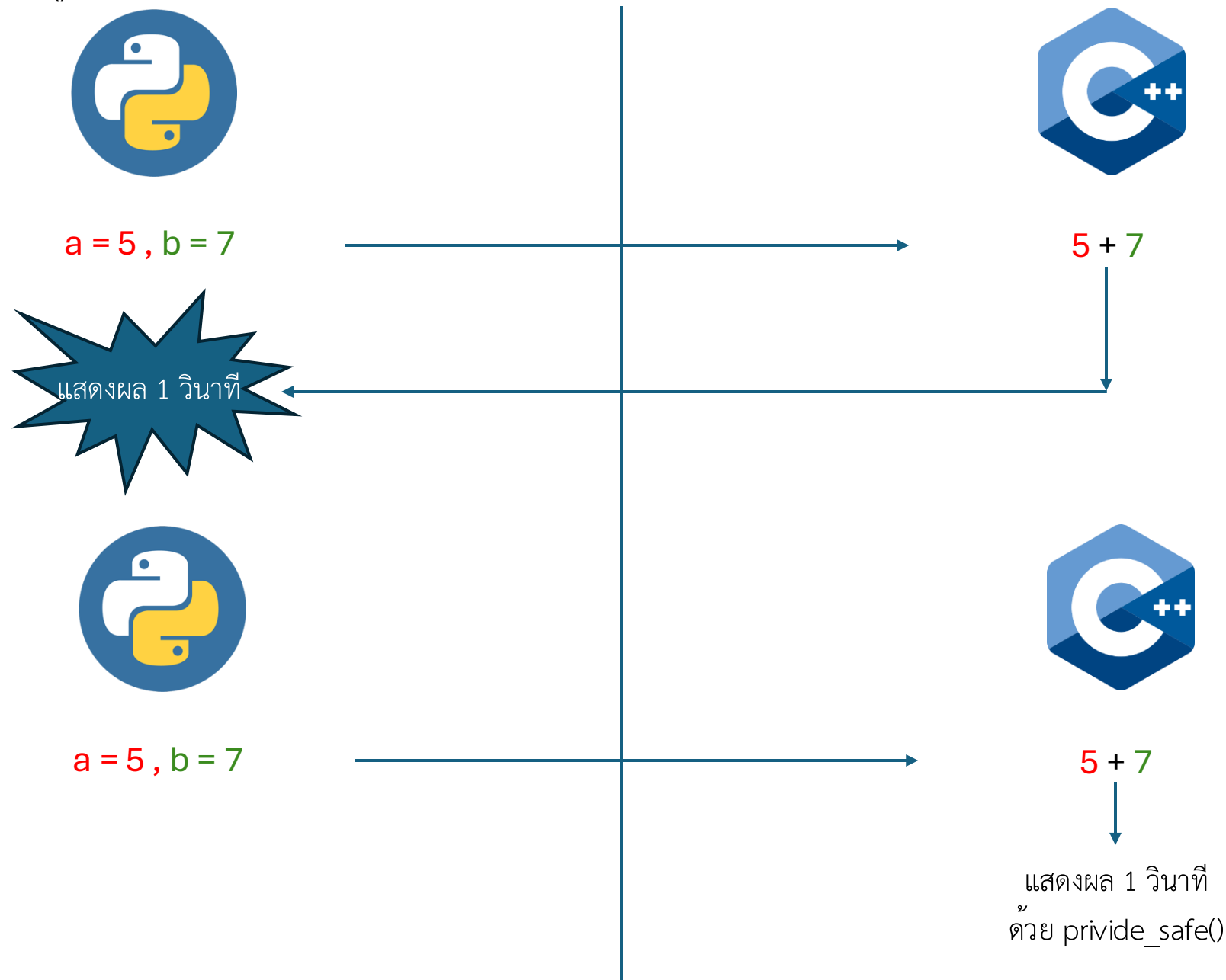


แสดงผล 1 วินาที

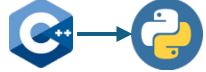


notify() and provide_safe()

return



call() and provide()



โยนค่ามากกว่า 1 ค่า $x = 10, y = 5$

$10 - 5$

แสดงผล 1 วินาที

return

$a = 5, b = 7$

$5 + 7$





```
from arduino.app_utils import App, Bridge
import time

a = 5

def loop():
    Bridge.call("Display_sum", a)

App.run(user_loop=loop)
```



```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Bridge.provide("Display_sum", sum);

    Monitor.begin();
}

void loop() {
    Monitor.println(z);
    delay(1000);
}

void sum(int x){
    z = x + 2;
}
```



```
from arduino.app_utils import App, Bridge
import time

a = 5
b = 7

def loop():
    Bridge.call("Display_sum", a, b)

App.run(user_loop=loop)
```



```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Bridge.provide("Display_sum", sum);

    Monitor.begin();
}

void loop() {
    Monitor.println(z);
    delay(1000);
}

void sum(int x, int y){
    z = x + y;
}
```



```
from arduino.app_utils import App, Bridge
import time

a = 5
b = 7
result = 0

def loop():
    result = Bridge.call("Display_sum", a, b)
    print(result)
    time.sleep(1)

App.run(user_loop=loop)
```



```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Bridge.provide("Display_sum", sum);
}

void loop() {

}

int sum(int x, int y){
    z = x + y;
    return z;
}
```



```
from arduino.app_utils import App, Bridge
import time

a = 5
b = 7
result = 0

def loop():
    result = Bridge.notify("Display_sum", a, b)
    print(result)
    time.sleep(1)

App.run(user_loop=loop)
```

```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Bridge.provide("Display_sum", sum);

    Monitor.begin();
}

void loop() {
    Monitor.println(z);
    delay(1000);
}

int sum(int x, int y){
    z = x + y;
    return z;
}
```



```
from arduino.app_utils import App, Bridge
import time

a = 5
b = 7
result = 0

def loop():
    result = Bridge.call("Display_sum", a, b)
    print(result)
    time.sleep(1)

App.run(user_loop=loop)
```



```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Bridge.provide_safe("Display_sum", sum);

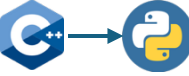
    Monitor.begin();
}

void loop() {

}

int sum(int x, int y){
    z = x + y;
    Monitor.println(z);
    delay(1000);
    return z;
}
```





```
#include "Arduino_RouterBridge.h"

void setup() {
  Bridge.begin();
}

void loop() {
  int x = 10;
  int y = 5;

  Bridge.call("Display_sub", x, y);
}
```



```
from arduino.app_utils import App, Bridge
import time

bridge = Bridge()

c = 0

def sub(a, b):
    global c
    c = a - b

bridge.provide("Display_sub", sub)

def loop():
    print(c)
    time.sleep(1)

App.run(user_loop=loop)
```

ในฝั่งไพธอนไม่มีคำสั่ง provide_safe



```
Activating python virtual environment
```

```
Traceback (most recent call last):
```

```
File "/app/python/main.py", line 12, in <module>
```

```
bridge.provide_safe("Display_sub", sub)
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
AttributeError: 'Bridge' object has no attribute 'provide_safe'
```

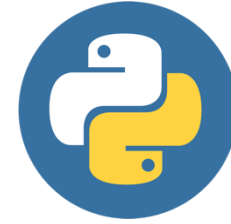


```
#include "Arduino_RouterBridge.h"

void setup() {
    Bridge.begin();
}

void loop() {
    int x = 10;
    int y = 5;

    Bridge.call("Display_sub", x, y);
}
```



```
from arduino.app_utils import App, Bridge
import time

bridge = Bridge()

c = 0

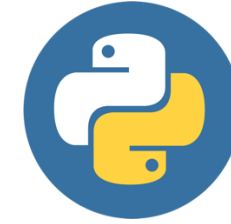
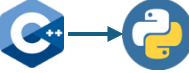
def sub(a, b):
    global c
    c = a - b
    print(c)
    time.sleep(1)

bridge.provide("Display_sub", sub)

def loop():
    pass

App.run(user_loop=loop)
```

ในฝั่งไพธอนไม่มีการรีเทิร์นค่ากลับไปฝั่ง c++ โดยตรง ให้คิดว่า call
ฝั่ง c++ เหมือนกับ notify



```
#include "Arduino_RouterBridge.h"

int z = 0;

void setup() {
    Bridge.begin();
    Monitor.begin();
}

void loop() {
    int x = 10;
    int y = 5;
    z = Bridge.call("Display_sub", x, y);
    Monitor.println(z);
    delay(1000);
}
```

```
from arduino.app_utils import App, Bridge
import time

bridge = Bridge()

c = 0

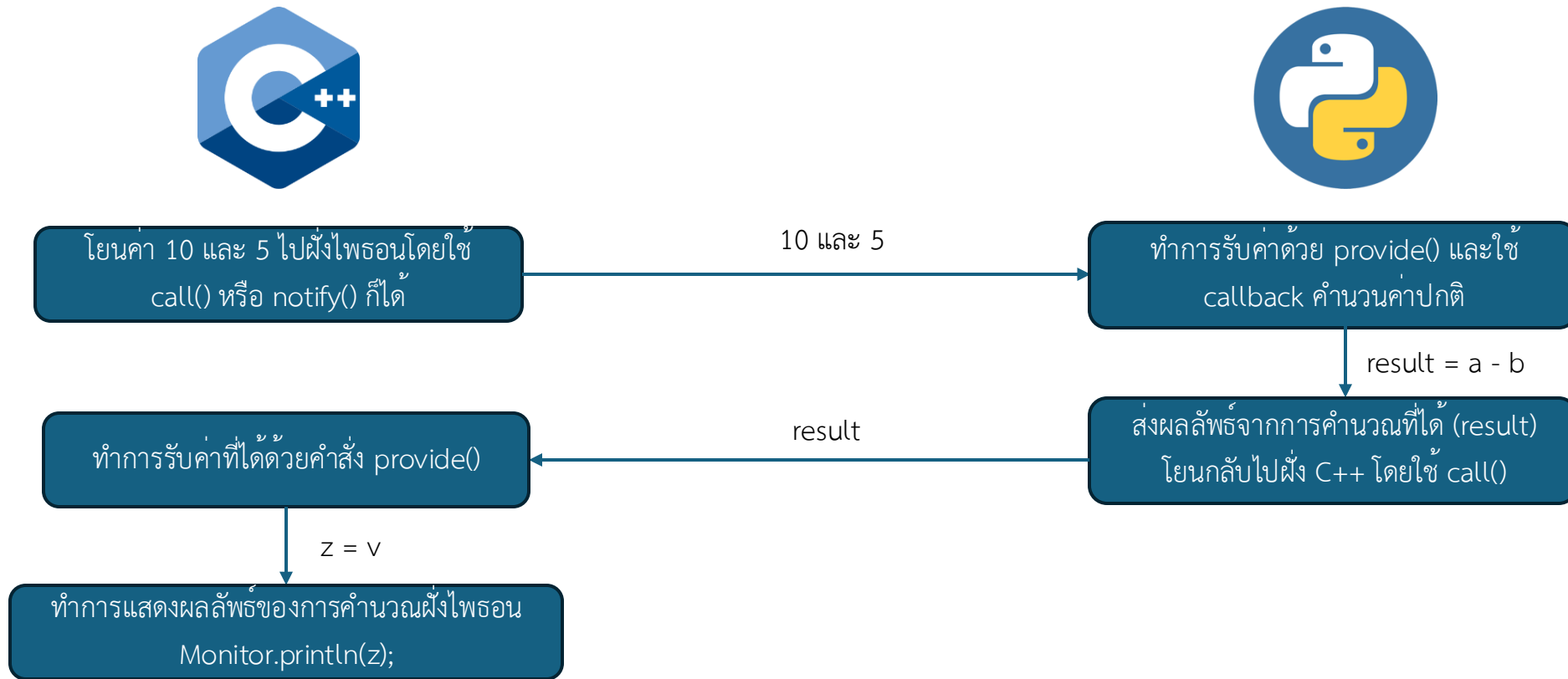
def sub(a, b):
    global c
    c = a - b
    return c

bridge.provide("Display_sub", sub)

def loop():
    print(c)
    time.sleep(1)

App.run(user_loop=loop)
```

วิธีแก้ไขคือ ให้ฝั่ง C++ โยนค่าไปก่อน เมื่อฝั่งไพธอน ทำการคำนวณเสร็จแล้ว ค่อยให้ไพธอนโยนค่ากลับมา





```
#include "Arduino_RouterBridge.h"

int z = 0;

void on_result(int v) {
    z = v;
}

void setup() {
    Bridge.begin();
    Bridge.provide("Return_sub", on_result);
    Monitor.begin();
}

void loop() {
    Bridge.call("Display_sub", 10, 5);
    Monitor.println(z);
    delay(1000);
}
```



```
from arduino.app_utils import App, Bridge
import time

bridge = Bridge()

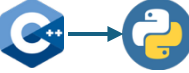
pending = False
result = 0

def sub(a, b):
    global pending, result
    result = a - b
    pending = True      # ตั้งธง

bridge.provide("Display_sub", sub)

def loop():
    global pending
    if pending:
        bridge.call("Return_sub", result)
        pending = False
        time.sleep(0.1)

App.run(user_loop=loop)
```



ฝั่ง c++ ใช้ notify ก็ได้เหมือนกัน (เพราะเหมือนกัน)



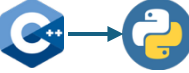
```
#include "Arduino_RouterBridge.h"

int z = 0;

void on_result(int v) {
    z = v;
}

void setup() {
    Bridge.begin();
    Bridge.provide("Return_sub", on_result);
    Monitor.begin();
}

void loop() {
    Bridge.notify("Display_sub", 10, 5);
    Monitor.println(z);
    delay(1000);
}
```



```
from arduino.app_utils import App, Bridge
import time

bridge = Bridge()

pending = False
result = 0

def sub(a, b):
    global pending, result
    result = a - b
    pending = True      # ตั้งธง

bridge.provide("Display_sub", sub)

def loop():
    global pending
    if pending:
        bridge.call("Return_sub", result)
        pending = False
        time.sleep(0.1)

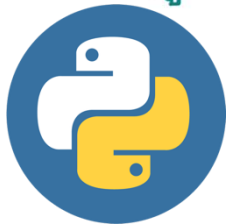
App.run(user_loop=loop)
```



Call (เจ้านาย)



Provide (ลูกน้อง)



`begin()`

`call()`

`notify()`

`provide()`

~~`provide_safe()`~~

} มีค่าเหมือนกัน

สรุปในกรณีที่ฝั่ง C++ เป็นเจ้านายและฝั่งไพธอนเป็นลูกน้อง

- ฝั่ง C++ ไม่สามารถรับการรีเทิร์นค่าจากฝั่ง Python ได้
- คำสั่ง `call()` และ `notify()` มีค่าเท่ากัน
- ที่ฝั่งไพธอนไม่มีออปเจกที่ชื่อว่า `provide_safe()` เนื่องจากไม่จำเป็นเนื่องจากฝั่งลีนุกซ์มีทรัพยากรในการใช้งานเหลือเฟือ